

DBMS

mm: 8-10

⇒ Syllabus :-

* Integrity Constraints & ER model

* Normalization [Schema Refinement]

* Queries

→ Relational Algebra

→ SQL

→ Relational Calculus

* Indexing

* Transactions & Concurrency Control

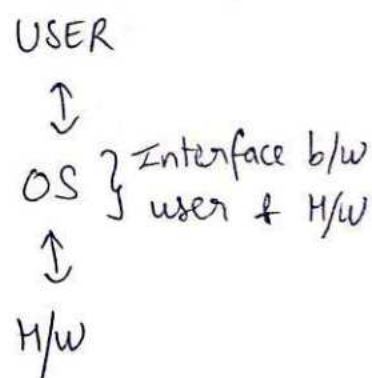
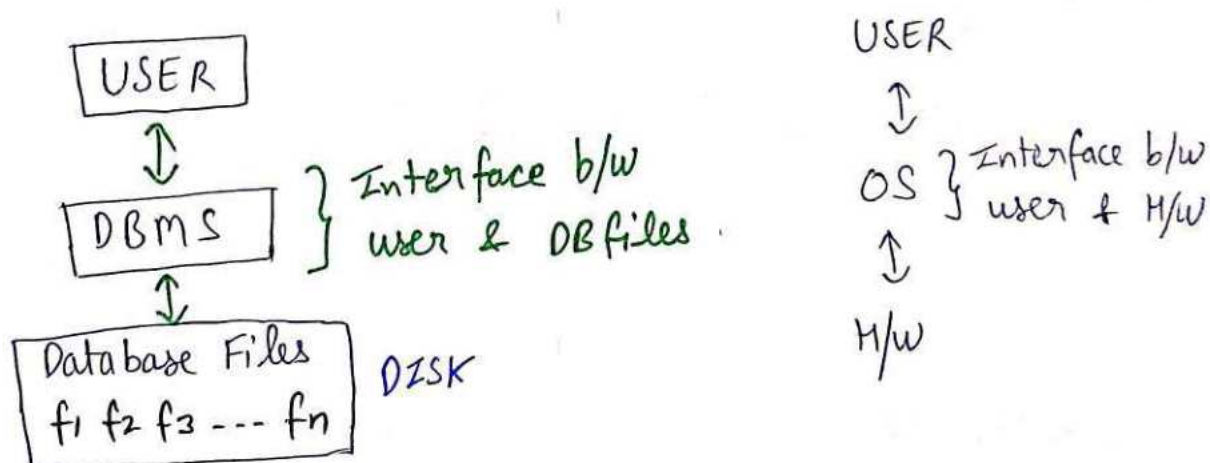
* Introduction :-

* Database :- A database is an organized collection of data which is stored & accessed from a computer system.

Eg:- University Database which includes [Collection of Related Data]

- Students info
- Faculty info
- Courses info
- Class rooms info etc.

* DBMS :- Software designed to define, manipulate, retrieve large collection of data in a Database. Access



* Flat File System [os files] :-

⇒ Database store in files are managed without using DBMS software



- ⇒ DB files must manage by user [format of file, format of data manages by user]
- ⇒ Data can store in any format based on user criteria.
- ⇒ Suitable to manage small DB failed to manage large DB

Eg:- [.txt, .csv etc.]

Stu.csv

S1, A, 20 # S2, B, 21 # S3, A, 22 #

(Flat File System)

* Limitations of Flat File System

- 1) Too complex to manage application programs & Too complex to develop application programs.
- 2) More I/O cost (access cost) to access data required from DB files
- 3) Less degree of concurrency
of simultaneous users can use DB
- 4) Complex to manage non-redundant DB
- 5) Difficult to implement different levels of access control (security)

Adv. of DBMS File System

- 1) Because of data independency easy to develop application program & easy to manage application programs. [SQL query]
- 2) Because of DB indexing reduces I/O cost to access required data from DB files.
- 3) more degree of concurrency.
- 4) Easy to maintain non-redundant data using normalization of DB.
- 5) By using view's [virtual tables] easy to implement different levels of access control.

* Integrity Constraints :-

- Candidate Key
- Primary Key
- Alternative Key
- Super Key
- Foreign Key etc.

* Data Model :- Data model define how the logical structure of a DB is modeled.

⇒ Data Models :-

- ✓ ① Relational Data Model [RDBMS] ← widely used ✓
- ② Object oriented data model [OODBS]
- ③ Hierarcial Data model
- ④ Network Data model

* RDBMS :- { EF Codd }

→ Codd proposed [12 Rules] for development of RDBMS S/W
RDBMS Guidelines

→ Codd's Data model

* RDBMS :- ① S/W which is used to store data which needs to be stored in the tabular form. [set of Rows & columns]
[Data managed & stored in rows & columns]

Eg:- Student Relation Attribute

Instance (Snapshot)

Stud	Sid	Sname	age
	S01	A	20
	S02	B	20
	S03	C	25
	S04	A	20

RDBMS File

Record/Tuple

⇒ Relational Instance [Snapshot], Record Set of DB Table.

⇒ Relational Schema :- Definition of DB Table

{ Stud (Sid, Sname, age) } Name of Table & Attributes

⇒ Arity :- Number of attributes of DB Table.

⇒ Cardinality :- Number of records of DB Table.

⇒ No two Rows of RDBMS table should be same.

* Candidate Key :-

⇒ minimal Attribute set which can differentiate records of relation uniquely.

Eg:- ① Stud (Sid Sname age) Candidate Key: Sid.

S1 A
S2 B
S3 B
S4 A
~~S4~~

SidSname: Not candidate Key

② Enroll

<u>Sid</u>	<u>Cid</u>	fee
S1	C1	4000
S1	C2	5000
S2	C2	5000
S3	C2	4000

Constraint :-

Student can enroll many courses
and
courses can be enroll by many
students.

SidCid : Candidate Key.

③ R

<u>A</u>	<u>B</u>	<u>C</u>
4	5	8
4	5	6
4	4	8
4	4	6
6	5	8

RDBMS Table

Candidate Key :- ABC

⇒ Relation can have many candidate keys.

Eg:-

Emp

<u>Eid</u>	Ename	DOB	<u>PANID</u>	<u>AdharID</u>	<u>IFSC Code</u>	<u>BANK ACC No.</u>
e1	A			501	501	101
e2	B			NULL	101	101
e3	B			505	501	102
e4	A			NULL	101	102

Assume No two Emp's with
same IFSC code & ACC No.

Candidate Keys :-

{ Eid, PANID, AdharID, IFSC BANK ACC }

Primary Key: Eid

Alternative Keys: PANID, AdharID, IFSC BANK ACC

* NULL :- Unknown / unexisted value

⇒

Primary Keys

- Any one candidate key whose field values not allowed NULL
- Field values not allowed NULL
- At most one primary key allowed for any relation

Alternative Keys

- All candidate keys of relation except primary key.
- Field values can be NULL's
- Many alternative keys allowed for relation.

* Schema Definition :-

⇒ Emp (Eid Ename DOB PANID AdharID IFSC Code ACC No.)

* CREATE TABLE EMP

```
( eid varchar(10) PRIMARY KEY,  
  ename varchar(30) NOT NULL,  
  dob date,  
  pan varchar(10) UNIQUE NOT NULL,  
  adhar integer(16) UNIQUE,  
  ifsc varchar(6),  
  ano. integer(10),  
  UNIQUE (ifsc, ano)  
);
```

Data Types

char
varchar
integer
date
timestamp etc.

* Simple Candidate Key:- Candidate Key with only one attribute.

* Compound Candidate Key:- Candidate key with at least two attributes.

* Prime Attribute :- Attribute which belongs to some candidate atleast one (Key attributes) key of relation.

Eg:- Emp (Eid Ename DOB PANID AdharID IFSC ACC
code No.)

→ Candidate Keys { eid, PANID, AdharID, ifsc code acc no }

→ Simple Candidate Keys : { eid, PANID, adharID }

→ Compound Candidate Keys : { ifsc Acc }

→ Prime Attribute set of Emp : { eid, PANID, adharID, ifsc code, ACC NO. }

→ Non-Prime Attribute set of Emp : { ename, DOB }

* Every RDBMS relation there must be atleast one candidate key whose field values not NULL.

* Primary Key not mandatory for RDBMS relation.

Eg:-

① CREATE TABLE R

(A int UNIQUE,
B int UNIQUE,
C int

)
R(A, B, C)
not allowed in RDBMS

② CREATE TABLE R

(A int UNIQUE NOT NULL,
B int UNIQUE,
C int

)
R(A, B, C)
allowed in RDBMS

③ CREATE TABLE R
(preferred)

(A int PRIMARY KEY,
B int UNIQUE,
C int

)
R(A, B, C)
allowed in RDBMS

* Super Key :- Set of attributes which can differentiate records of relation uniquely. [may not minimal Attr set]

⇒ Candidate Key : Minimal Super Key

⇒ Every candidate Key is superkey, Not every superkey is candidate Key.

Set of Candidate
Keys of Relation (R)

Set of Super Keys
of Relation (R)

⇒ Student (Sid Sname age)

S1 A
S2 A
S3 B
S4 B
S5 B

Candidate Key : Sid

Super Keys : { Sid ,
(4 SK's) Sid Sname ,
Sid age ,
Sid Sname age }
minimal
SK: CK

1) How many super keys in relation R(A, B, C, D) with Candidate Key {A}.

Soln R(A ^{2³}BCD) SK's : { A AB ABC ABCD
AC ABD
AD ACD }

of SK's : 8 //

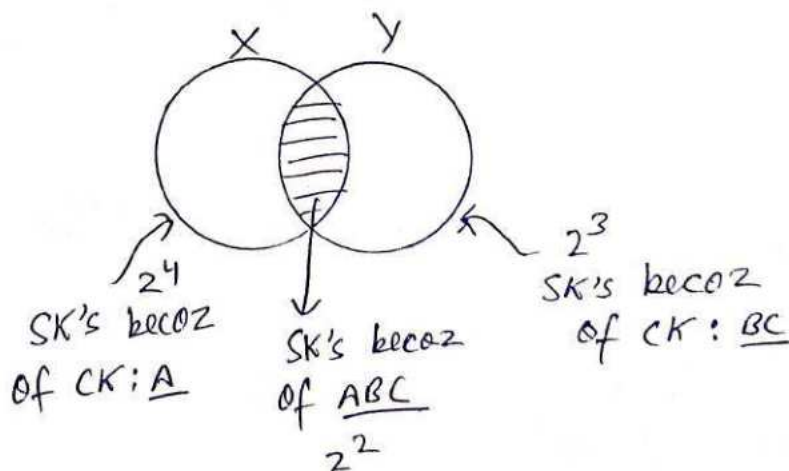
2) How many super keys in relation R(A, B, C, D, E) with

i) Candidate Key { A, BC }

Soln A ^{2⁴}[BCDE]
BC ^{2³}[ADE]

ABC
ABCD
ABCE
ABCDE

ABC ^{2²}[DE]

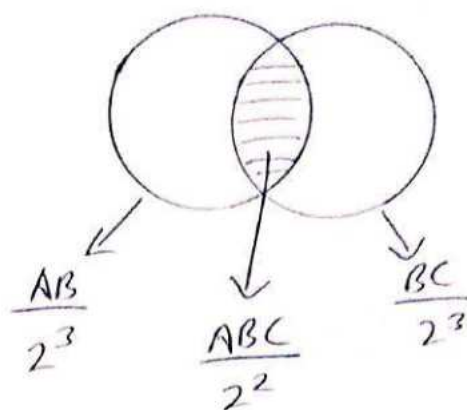


$$n(X \cup Y) = n(X) + n(Y) - n(X \cap Y)$$

$$= 16 + 8 - 4 = 20 // \text{SK's}$$

ii) Candidate keys : $\{ \underline{AB}, \underline{BC} \}$

Sepn
 $\underline{AB} [CDE]_{2^3}$
 $\underline{BC} [ADE]_{2^3}$

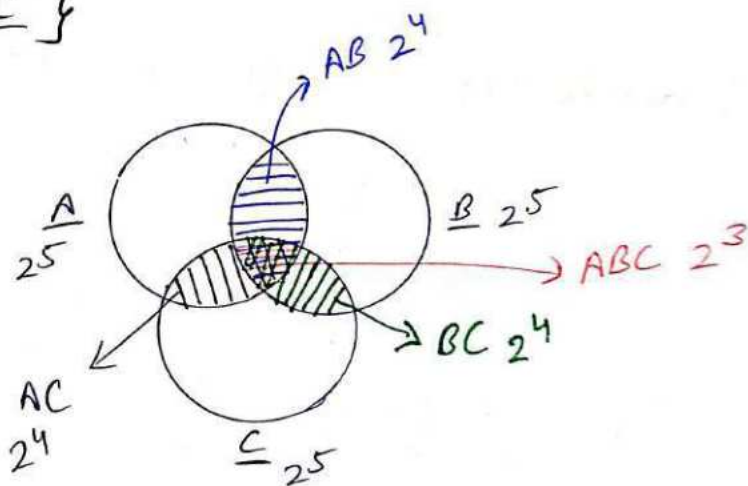


$$\Rightarrow 8 + 8 - 4 = 12 \text{ SK's}$$

3) How many super keys in relation $R(A, B, C, D, E, F)$

i) Candidate Key $\{ \underline{A}, \underline{B}, \underline{C} \}$

Sepn
 $\underline{A} [BCDEF]_{2^5}$
 $\underline{B} [ACDEF]_{2^5}$
 $\underline{C} [ABDEF]_{2^5}$



$$\Rightarrow n(X \cup Y \cup Z) = n(X) + n(Y) + n(Z) - n(X \cap Y) - n(Y \cap Z) - n(X \cap Z) + n(X \cap Y \cap Z)$$

$$= 3 \times 2^5 - 3 \times 2^4 + 2^3$$

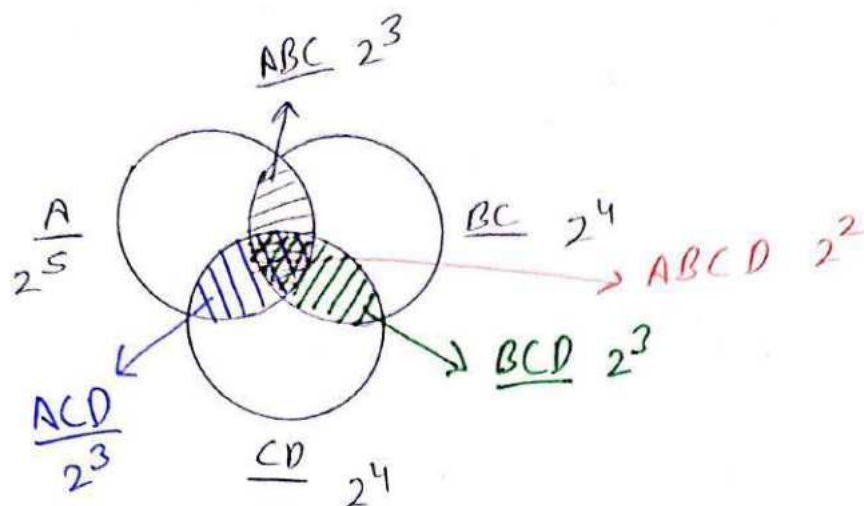
$$= 96 - 48 + 8$$

$$= 48 + 8$$

$$= 56 // \text{SK's}$$

ii) Candidate Keys : $\{ \underline{A}, \underline{BC}, \underline{CD} \}$

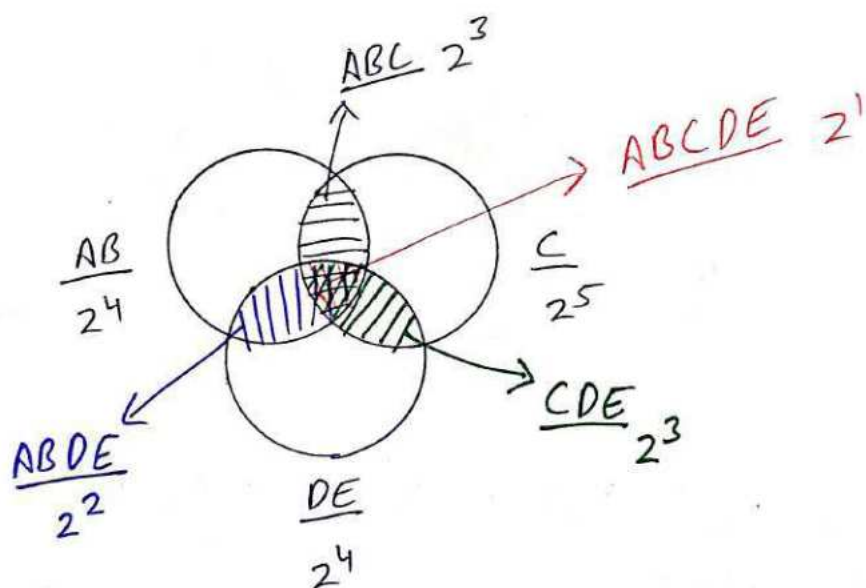
Soln



$$\Rightarrow 2^4 + 2^4 + 2^5 - 3 \times 2^3 + 2^2 = 44 \text{ SK'S}$$

iii) Candidate Keys : $\{ \underline{AB}, \underline{C}, \underline{DE} \}$

Soln

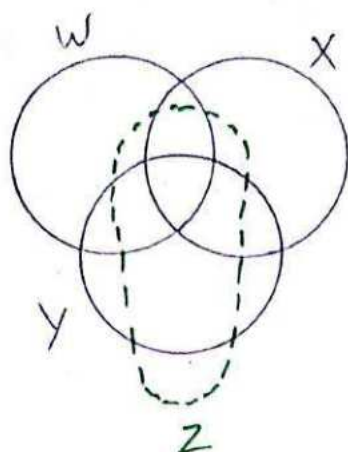


$$\Rightarrow 2^4 + 2^4 + 2^5 - 2^3 - 2^3 - 2^2 + 2^1$$

$$= 46 \text{ SK'S}$$

4) How many superkeys in relation R(A, B, C, D, E) with candidate keys : $\{ \underline{AB}, \underline{BC}, \underline{CD}, \underline{DE} \}$.

Soln



$$\Rightarrow n(W \cup X \cup Y \cup Z) = n(W) + n(X) + n(Y) + n(Z) - n(W \cap X) - n(W \cap Y) - n(W \cap Z) - n(X \cap Y) - n(X \cap Z) - n(Y \cap Z) + n(W \cap X \cap Y) + n(W \cap X \cap Z) + n(W \cap Y \cap Z) + n(X \cap Y \cap Z) - n(W \cap X \cap Y \cap Z) \dots$$

$$\Rightarrow \left\{ \begin{array}{l} AB \\ ABC \\ ABD \\ ABE \\ ABCD \\ ABCE \\ ABDE \\ ABCDE \\ [AB]CDE \end{array} \right\} \left\{ \begin{array}{l} BC \\ BCD \\ BCE \\ BCDE \\ A[BC]DE \end{array} \right\} \left\{ \begin{array}{l} CD \\ ACD \\ CDE \\ ACDE \\ AB[CD]E \end{array} \right\} \left\{ \begin{array}{l} DE \\ ADE \\ BDE \\ ABC[DE] \end{array} \right\} \left. \vphantom{\begin{array}{l} AB \\ ABC \\ ABD \\ ABE \\ ABCD \\ ABCE \\ ABDE \\ ABCDE \\ [AB]CDE \end{array}} \right\} 19 \text{ SK's}$$

5) $R(A_1, A_2, \dots, A_n)$ How many possible Superkeys in Relation R?

a) $n!$

b) 2^{2^n}

c) $2^n - 1$ ✓

d) 2^n

Soln

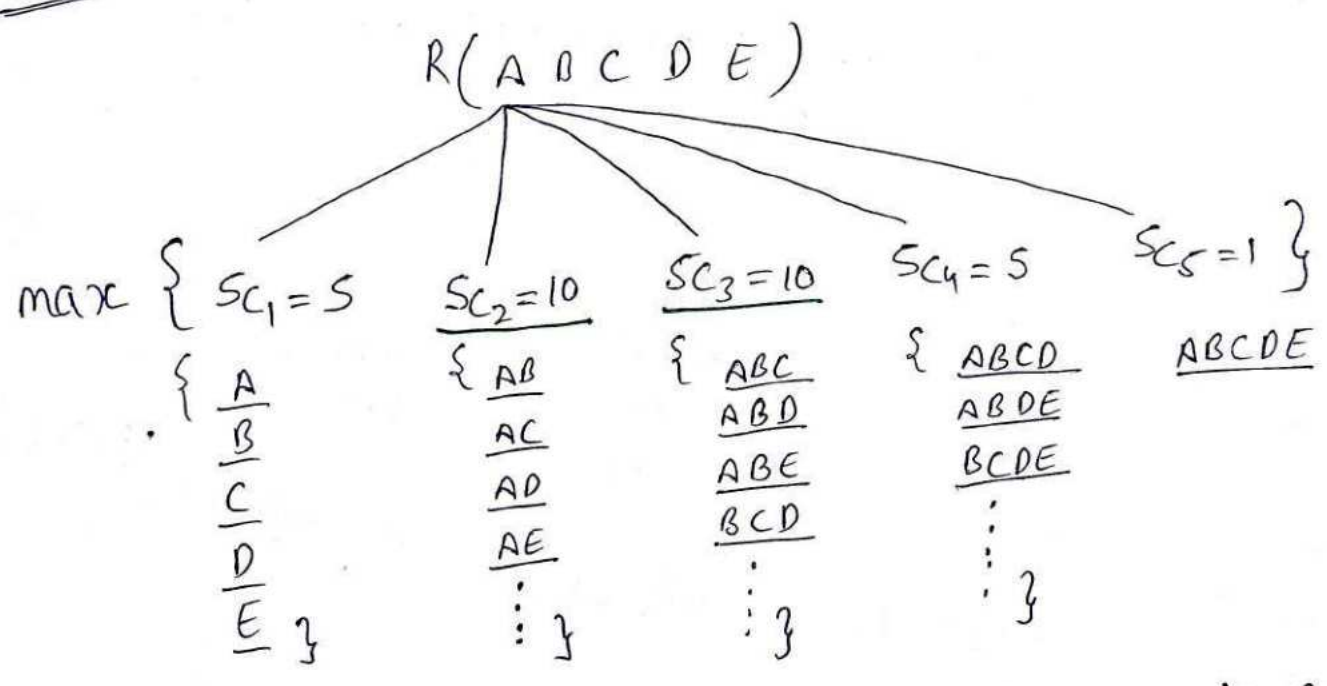
$R(\underline{A} \underline{B} \underline{C})$ SuperKey
 $2^3 - 1 = 7$
 $\{ A, B, C, AB, AC, BC, ABC \}$

$R_1(\underline{A_1} \underline{A_2} \underline{A_3} \dots \underline{A_n})$
 $2^n - 1$ Superkeys in Rel R
if each attribute of R
is candidate key.

6) $R(ABCDE)$ How many maximum candidate keys possible in R?

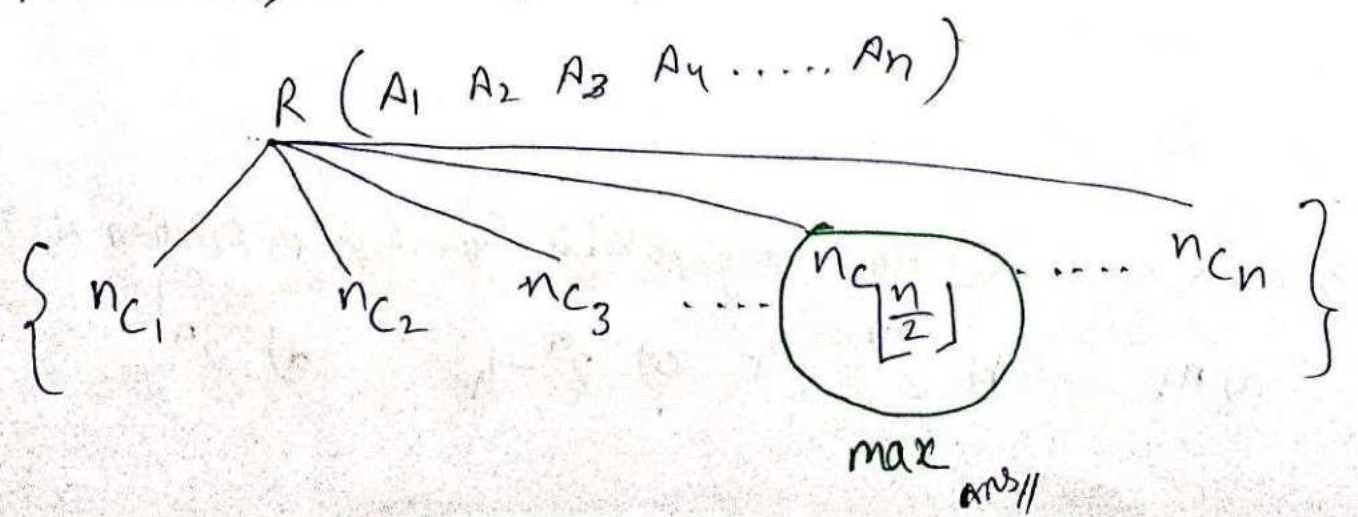
Soln

max : 10 CK's possible



7) $R(A_1 A_2 \dots A_n)$ How many maximum possible CK's in Rel R?

Soln



* Foreign Key :- [Referential Key]

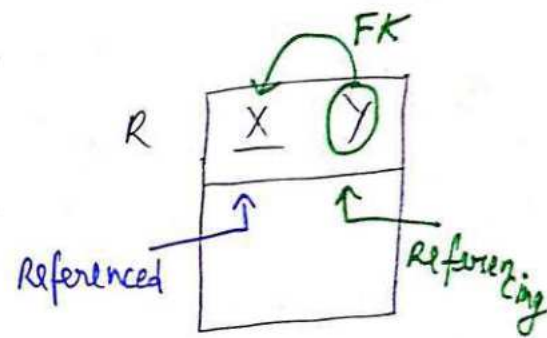
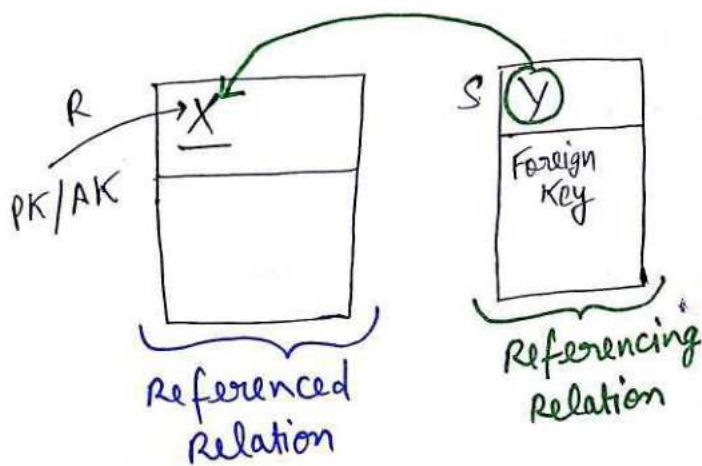
⇒ used to relate data [Relationship Set]

⇒ Defined over two relations

① Referenced Relation

② Referencing Relation

⇒ Definition :- Foreign Key is set of attributes references to primary key or alternative key of same relation or some other relation.



⇒ Referential Key Integrity Constraints :-

① Referenced Relation

Stud	<u>Sid</u>	Sname	DOB
	S1	A	—
	S2	A	—
	S3	B	—
	S4	A	—
	S5	C	—
	S6	B	—

Sid : Primary Key

② Referencing Relation

Enroll	<u>Sid</u>	<u>Cid</u>	fee
	S1	C1	5000
	S1	C2	4000
	S2	C2	5000
	S3	C1	6000
	S6	C2	4000
	S9	C1	5000

SidCid : Primary Key

→ S9 not in Stud Rel

① Insertions :- No Violation

② Deletion :- may cause FK violation

i) ON Delete NO Action (default)

→ Deletion Restricted if FK violation occurs

ii) ON Delete Cascade

→ Allowed to delete & DBMS deletes all related referencing records

iii) ON Delete Set NULL / Default

→ If FK values allowed to set NULL then to delete referenced record & sets NULL in related FK values.

③ Updation :- may cause FK violation

i) ON Update NO Action (default)

ii) ON Update Cascade

iii) ON Update Set NULL

① Insertion :-

may cause FK violation

② Deletion :-

No FK violation

③ Updation :-

may cause FK violation

* NOTE :-

① Foreign Key can reference primary key / alternative key of same Rel..

Emp

<u>eid</u>	<u>ename</u>	<u>supID</u> ^{FK}
e1	A	NULL
e2	B	e1
e3	C	e1
e4	D	e2
e5	D	NULL
e6	B	e5

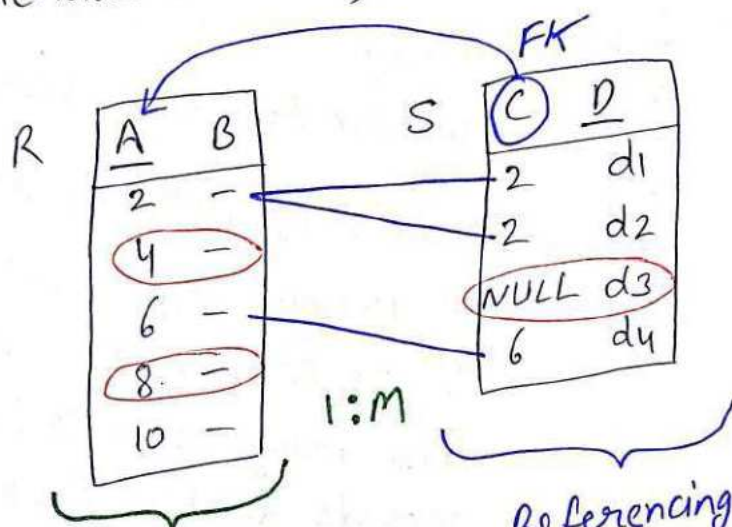
{ supervisor is also Emp }

Q) If FK constraint on delete cascade
How many records forced to delete if $eid = e_1$ deleted?

Ans = 3

② ⇒ Foreign Key allowed NULL values.

⇒ Referencing relation records whose foreign key fields value's null are out of referential (Not related to referential records)



mapping b/w
referential &
referencing records
is 1:M

Referenced Relation

⇒ Each record of referenced relation relates to many (0 or more) records of Referencing relation

Referencing Relation :

⇒ Each record of referencing relation related by atmost one (0 or 1) records of referenced relation.

③ If $R(ABC)$ with AB primary Key

Is allowed to create relation $S(C, D, E)$ with "C" foreign key references to R?

$R(\underline{A} \underline{B} C)$

$S(\underline{C} D E)$
FK

NOT allowed

④ If $R(ABC)$ with AB primary key
is allowed to create relation $S(C,D,E)$ with "CD"
foreign key references to R?

$R(\underline{ABC})$ $S(\underbrace{CDE}_{FK})$ Allowed

Q) Consider given relations

Hand-drawn diagram illustrating a join operation between two tables, R and S, using a join tree.

Table R:

A	B
2	3
4	3
6	4
7	5
8	4
9	7
10	7

Table S:

C	D
2	2
3	2
6	4
4	4
8	7
5	9
7	9

Join Tree:

The join tree shows R and S connected by a join operator. The result of the join is shown as a table with columns A, B, C, and D.

Result Table:

A	B	C	D
2	3	2	2
4	3	6	4
6	4	4	4
7	5	8	7
8	4	5	9
9	7	7	9

Annotations:

- R.ed**: Points to the first column (A) of the result table.
- R.ing**: Points to the second column (B) of the result table.
- Set NU**: Points to the last two columns (C and D) of the result table.
- ON Rel Cascade**: Points to the result table.

In $R(A, B)$ with A Primary Key & B Foreign key references to S with ON DELETE CASCADE.

In $S(C, D)$ with C primary key
 & D foreign key references to A
 with ON DELETE SET NULL

How many total additional records deleted from Rel R & S when tuple (2,3) of Rel R deleted?

Ans = 0 //

* Normalization [Schema Refinement]

⇒ used to reduce/eliminate redundancy in DB relations.

⇒ Redundancy in relation because of two or more independent relations stored in single relation.

Eg:-

Sid → Sname age
(1)

Cid → Cname Inst
(2)

Sid Cid → fee
(3)

R

Sid	Sname	age	Cid	Cname	Inst	fee
S1	A	20	C1	DB	Karth	5000
S2	A	20	C1	DB	Karth	6000
S3	B	22	C1	DB	Karth	6000
S3	B	22	C2	OS	Galwin	6000
S3	B	22	C3	CO	Hayes	4000

Sid Cid :
Primary Key

Redundancy

* Problems because of Redundancy (DB Anomalies) :-

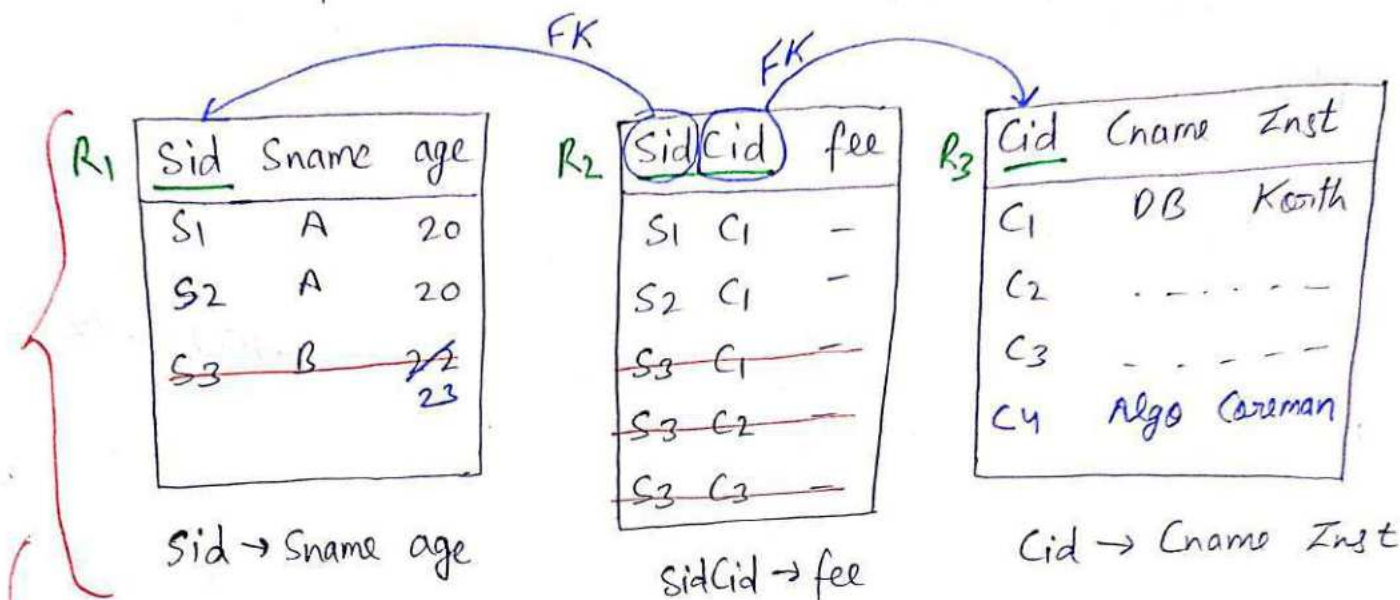
① Insertion Anomaly :- To insert some data forced to insert other independent data.

② Deletion Anomaly :- Because of deletion of some data forced to delete other independent data.

③ Updation Anomaly :- some redundant data updated other redundant copy failed to update.

* Decomposition :-

⇒ Split Rel R into two or more sub relations to reduce / eliminate redundancy / DB anomalies.



Normalized
DB design

[No Redundancy & No Anomalies]

* Functional Dependency :- ($X \rightarrow Y$)

⇒ X, Y some attribute sets over R.

$X \rightarrow Y$ exists in R

if $t_1 \cdot X = t_2 \cdot X$ then $t_1 \cdot Y = t_2 \cdot Y$

R

X	Y
4	8
4	8
4	8
2	6
3	6
7	5

$X \rightarrow Y$
exists in R

R

X	Y...
2	5
4	5
4	6
2	6

$X \rightarrow Y$
not implied in R

R

A	B	C	D
2	4		
3	4		
4	7		
5	8		
6	9		

$A \rightarrow B$
 $A \rightarrow C$
 $A \rightarrow D$

} Implied in R

$\Rightarrow$ Trivial FD :- (Reflexive FD's)

$X \rightarrow Y$ is trivial FD
if $X \supseteq Y$

Eg:-

$Sid \rightarrow Sid$
 $Sname \rightarrow Sname$
 $Cid \rightarrow Cid$
 $Sid\ Sname \rightarrow Sname$
 $Sid\ Sname \rightarrow Sid\ Sname$

R

Sid	Sname	Cid
S1	A	C1
S1	A	C2
S2	B	C2
S2	B	C1
S3	B	C2

$\Rightarrow$ Every possible trivial FD of rel R is always implied in Rel R.

$\Rightarrow$ Non Trivial FD :-

$X \rightarrow Y$ non trivial FD

if no common attribute b/w X & Y attribute sets.

Eg:-

$Sid \rightarrow Sname$
 $Sid\ Cid \rightarrow Sname$

$AB \rightarrow C$
 $A \rightarrow BC$

}

⇒ Semi - Non trivial FD :-

Combination of trivial & Non-trivial FD's.

Eg:- • $Sid \rightarrow Sid\ Sname$

$$\begin{cases} Sid \rightarrow Sid \\ Sid \rightarrow Sname \end{cases}$$

$$\begin{cases} X \rightarrow YZ \Rightarrow \\ X \rightarrow Y, X \rightarrow Z \end{cases}$$

• $Sid\ Cid \rightarrow Cid\ Sname$

$$\begin{cases} Sid\ Cid \rightarrow Cid \\ Sid\ Cid \rightarrow Sname \end{cases}$$

• $AB \rightarrow AC$

$$\begin{cases} AB \rightarrow A \\ AB \rightarrow C \end{cases}$$

Q) ① Find Non-trivial FD's for given Instance

R

A	B	C
2	3	5
2	4	5
3	3	6
3	4	6

Soln

$$\begin{cases} \cancel{X}A \rightarrow B & \cancel{X}B \rightarrow A & \checkmark C \rightarrow A \\ \checkmark A \rightarrow C & \cancel{X}B \rightarrow C & \checkmark C \rightarrow B \\ \cancel{X}A \rightarrow BC & \cancel{X}B \rightarrow AC & \checkmark C \rightarrow AB \\ \checkmark AB \rightarrow C & & \\ \checkmark BC \rightarrow A & & \\ \cancel{X}AC \rightarrow B & & \\ & & \end{cases}$$

$$\{A \rightarrow C, C \rightarrow A, AB \rightarrow C, BC \rightarrow A\} //$$

② Find Non-trivial FD's

R

A	B	C
4	4	3
4	4	5
4	6	3
4	6	5
6	4	3

No. non-trivial FD's //

⇒ Armstrong Rules over FD's :-

X, Y, Z are some attribute sets over Relation(R)

① Reflexivity :- $X \rightarrow X$ (always True)
[Trivial FD]

$R(\dots\dots)$

$R(ABC)$

$A \rightarrow A$

$B \rightarrow B$

$C \rightarrow C$

② Transitivity :- If $X \rightarrow Y$ & $Y \rightarrow Z$
then $X \rightarrow Z$

$\{A \rightarrow B, B \rightarrow C\} \Rightarrow \{A \rightarrow C\}$

③ Augmentation :- If $X \rightarrow Y$ Then
 $XZ \rightarrow YZ$

$\{A \rightarrow B\} \Rightarrow \{AC \rightarrow BC\}$

④ Union/Merge Rule :- If $X \rightarrow Y$ &
 $X \rightarrow Z$ then $X \rightarrow YZ$

$\{A \rightarrow B, A \rightarrow C\} \Rightarrow \{A \rightarrow BC\}$

⑤ Split Rule :- If $X \rightarrow YZ$ then
 $X \rightarrow Y, X \rightarrow Z$

$\{A \rightarrow BC\} \Rightarrow \{A \rightarrow B, A \rightarrow C\}$

$\{AB \rightarrow C\} \not\Rightarrow \{A \rightarrow C, B \rightarrow C\}$
not allowed

Q) If $AB \rightarrow C$ then $A \rightarrow C, B \rightarrow C$ [TRUE / False]

* Attribute Closure :- $[X^+]$

$X^+ = \left\{ \text{set of all attributes which are} \right.$
 $\left. \text{determined by } X \right\}$

Eg:- Given FD Set $\{A \rightarrow B, CE \rightarrow F, B \rightarrow C, BD \rightarrow E, FG \rightarrow H\}$

- $A^+ = \{A B C\}$ $A \rightarrow A B C$
- $(AD)^+ = \{A D B C E F\}$ $AD \rightarrow A B C D E F$
- $(ADG)^+ = \{A D G B C E F H\}$ $ADG \rightarrow A B C D E F G H$

* Membership Test :-

$\Rightarrow X \rightarrow Y$ FD is member of FD set (F)
iff X^+ must determine Y in FD set (F)

$$F = \{\dots\dots\} \leftarrow \underbrace{(X \rightarrow Y)}_{\text{member of}} \quad X^+ = \{\dots\dots Y \dots\dots\}$$

Q) Consider given FD set $\{AB \rightarrow CD, AF \rightarrow D, DE \rightarrow F, C \rightarrow G, F \rightarrow E, G \rightarrow A\}$ which FD is not member of given FD set?

- a) $BEG \rightarrow CF$ b) $BG \rightarrow D$
c) $BCF \rightarrow A$ d) $AB \rightarrow E$ $(AB)^+ = \{A B C D G\}$

* F & G Logically equal :-

$\Rightarrow F$ & G FD sets are logically equal iff

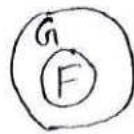
① F covers G :- Every FD of G set must member of F Set.



$$F \supseteq G$$

And

② G covers F :- Every FD of F set must member of G set.



$$F \subseteq G$$

$\Rightarrow$ ① F covers G

TRUE

TRUE

False

False

② G covers F

TRUE

False

TRUE

False

$$\Rightarrow F = G$$

$$\Rightarrow F \supset G$$

$$\Rightarrow F \subset G$$

$\Rightarrow F$ & G are not comparable

Q) Consider given FD sets

$$F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$$

$$\begin{aligned} A^+ &= \underline{ABC} \\ B^+ &= \underline{BCA} \\ (AB)^+ &= \underline{ABC} \end{aligned}$$

$$(BC)^+ = \underline{BCA}$$

$$G = \{A \rightarrow BC, B \rightarrow AC, AB \rightarrow C, BC \rightarrow A\}$$

$$\begin{aligned} A^+ &= \{\dots B\} \\ B^+ &= \{\dots C\dots\} \\ C^+ &= \{C\} \end{aligned}$$

which is TRUE for given FD set?

- a) $F \subset G$ b) $F \supset G$ c) $F = G$ d) None

soln F covers G (Yes) | G covers F (No)

Q) Consider given FD sets

$$F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$$

$$G = \{A \rightarrow BCD, B \rightarrow ACD, C \rightarrow ABD, BD \rightarrow A, AD \rightarrow B, AB \rightarrow D, BC \rightarrow A\}$$

which is true for given FD set?

- a) $F \subset G$ ☒ b) $F \supset G$ c) $F = G$ d) None

Q) Consider given FD sets

$$F = \{ A \rightarrow BCDEF, BC \rightarrow ADEF, D \rightarrow E, B \rightarrow F, AD \rightarrow E \}$$

$$G = \{ A \rightarrow BC, BC \rightarrow AD, D \rightarrow E, B \rightarrow F \}$$

which is true for given FD set?

- a) $F \subset G$ b) $F \supset G$ ☒ c) $F = G$ d) None

Soln

F covers G

$$A^+ = \{ \underline{A}BCDEF \}$$

$$BC^+ = \{ \underline{A}BC\underline{D}EF \}$$

$$D^+ = DE$$

(Yes)

G covers F

$$A^+ = \{ \underline{A}BCDEF \}$$

$$BC^+ = \underline{B}CADEF$$

$$AD^+ = \underline{A}BCDE\underline{F}$$

(Yes)

* Canonical cover or minimal cover :- (Non-reducible equal FD set)

⇒ Finding minimal possible FD's which are logically equivalent to given FD set.

Eg:- Given FD set $\{ A \rightarrow B, B \rightarrow C, \overset{X}{\underbrace{A \rightarrow C}}, \overset{X}{\underbrace{A \rightarrow A}}, \overset{X}{\underbrace{AB \rightarrow C}} \}$

Trivial

$B \rightarrow C \Rightarrow AB \rightarrow AC$

$\Rightarrow AB \rightarrow A$

$AB \rightarrow C$

$\rightarrow \{ A \rightarrow B, B \rightarrow C \}$ // minimal cover

⇒ Extraneous attribute Over determinant :- → LHS side of FD

$$\{ \underline{X} Y Z \rightarrow W, X \rightarrow Y \} = \{ XZ \rightarrow W, X \rightarrow Y \}$$

↙
↘

extraneous attr.
Determinant (LHS)
Determiner (RHS)

⇒ Remove extraneous attribute from determinant.

① $\{ \underline{AB} \rightarrow C, A \rightarrow B \} = \{ A \rightarrow C, A \rightarrow B \}$

↓

extraneous

② $\{ \underline{ABCD} \rightarrow E, A \rightarrow F, F \rightarrow B \} = \{ ACD \rightarrow E, A \rightarrow F, F \rightarrow B \}$

↓

extraneous

③ $\{ \underline{AB}^F \rightarrow C, B \rightarrow A, A \rightarrow B \} = \begin{cases} \{ B \rightarrow C, B \rightarrow A, A \rightarrow B \} \\ \{ A \rightarrow C, B \rightarrow A, A \rightarrow B \} \end{cases}$

④ $\{ \underline{ABCD} \rightarrow E, A \rightarrow D, BC \rightarrow A \} = \{ BC \rightarrow E, A \rightarrow D, BC \rightarrow A \}$

$BC^+ = \{ \dots AD \dots \}$

⇒ Redundant FD :- $X \rightarrow Y$ FD from FD set (F) is redundant iff $X \rightarrow Y$ must be member of $\{ F - (X \rightarrow Y) \}$ FD set.

Redundant
 $\{ \dots \cancel{X \rightarrow Y} \dots \}$
 $X^+ = \{ \dots Y \dots \}$

* Remove Redundant FD's :-

① $\{ A \rightarrow B, B \rightarrow C, \cancel{A \rightarrow C} \}$

↘

Redundant

$A^+ = \dots C \dots$

② $\{ A \rightarrow B, \cancel{AC \rightarrow B} \} = \{ A \rightarrow B \}$

$(AC)^+ = \{ \dots B \}$

③ $\{ AB \rightarrow C, \cancel{AB \rightarrow D}, BC \rightarrow D, \cancel{BC \rightarrow E}, CD \rightarrow E \} = \{ AB \rightarrow C, BC \rightarrow D, CD \rightarrow E \}$

$AB^+ = ABCDE$ $BC^+ = BCE$ $BC^+ = BCDE$

* Procedure to find minimal cover :-

- ⇒ Assume Given FD set F.
- ① Remove extraneous attributes from each determinant of given FD set.
- ② Remove Redundant FD's from result of step 1.

Q) Find minimal cover of Given FD set.

① $\{ A \rightarrow BC, D \rightarrow E, E \rightarrow C, D \rightarrow AEH, ABH \rightarrow BD, DH \rightarrow BC \}$

$D^+ = \{ \dots C \dots \}$
 $A^+ = \{ \dots B \dots \}$
 $D^+ = \{ \dots H \dots \}$

Split FD's ⇒

Soln 1) Remove extraneous attr. from each determinants

Split FD's

$\{ A \rightarrow B \checkmark, A \rightarrow C \checkmark, D \rightarrow E \checkmark, E \rightarrow C \checkmark, D \rightarrow A \checkmark, D \rightarrow H \checkmark, \cancel{D \rightarrow E}, \cancel{D \rightarrow H} \}$

$\{ \cancel{AH \rightarrow B} \times, AH \rightarrow D \checkmark, \cancel{D \rightarrow B} \times, \cancel{D \rightarrow C} \times \}$

2) Remove Redundant FD's

$\{ \dots X \rightarrow Y \dots \}$
 only one FD

$\{ \dots (X \rightarrow Y, W \rightarrow Y) \dots \}$

$\therefore$ minimal cover $\{ A \rightarrow BC$
 $D \rightarrow EAH$
 $E \rightarrow C$
 $AH \rightarrow D \}$

② Find minimal cover

$\{ A \rightarrow BCDEF$
 $BC \rightarrow ADEF$
 $B \rightarrow F$
 $D \rightarrow E$
 $A \rightarrow D \}$ $A^+ = \dots E \dots$

Soln 1) Remove extraneous attr.

$\Rightarrow$ Split FD's $\rightarrow$ 2) Remove Redundant FD's

$\{ A \rightarrow B \checkmark$ $BC \rightarrow A \checkmark$ $B \rightarrow F \checkmark$
 $A \rightarrow C \checkmark$ $BC \rightarrow D \checkmark$ $D \rightarrow E \checkmark \}$

$\times$ $A \rightarrow D$ $\times$ $BC \rightarrow E$
 $\times$ $A \rightarrow E$ $\times$ $BC \rightarrow F$
 $\times$ $A \rightarrow F$

$\Rightarrow$ minimal cover = $\{ A \rightarrow BC, BC \rightarrow AD, B \rightarrow F, D \rightarrow E \}$
 (OR) = $\{ A \rightarrow BCD, BC \rightarrow A, B \rightarrow F, D \rightarrow E \}$

NOTE :- Minimal cover of FD set F may not be unique.
 • All minimal covers of FD set F are logically equal.

$F = F_{m1} = F_{m2}$
 Given FD Set

* Super Key :-

$\Rightarrow$ X is super key of relation schema R iff X^+ must determine all attributes of relation R.

$$\left\{ \begin{array}{l} X \text{ is Super Key} \\ \text{of Rel R} \end{array} \right\} \Leftrightarrow \left\{ \begin{array}{l} X^+ \text{ determines all} \\ \text{attributes of Rel R} \end{array} \right\} \quad \left\{ \begin{array}{l} R(\dots\dots) \\ \text{FD set (F)} \end{array} \right\}$$

Eg:- $R(\overbrace{ABCDE}) \{AB \rightarrow C, C \rightarrow D, B \rightarrow E\}$

$$\text{SK} \rightarrow (AB)^+ = \{ABCDE\}$$

$$\text{SK} \rightarrow (ABC)^+ = \{ABCDE\}$$

$$\text{NOT SK} \rightarrow (BC)^+ = \{BCDE\}$$

* Candidate Key :- [Minimal Super Key]

$\Rightarrow$ X is candidate Key of relational Schema R iff

① X must be Super Key of R

$$\rightarrow X^+ = \{\text{all attributes of R}\}$$

(And)

② No proper subset of X is Super Key of R.

$\rightarrow \nexists Y \subset X$ such that Y^+ should not determine all attributes of R.

Imp***
Q) Find CK's of Relations :-

① $R(ABCD) \quad \{AB \rightarrow C\}$

Soln
 $\Rightarrow (ABD)^+ = \{ABCD\}$

ABD : Candidate Key //

② $R(ABC)$ with no non-trivial FD's.

Soln
Candidate Key : ABC //

$R \Rightarrow (ABC)^+ = \{ABC\}$

A	B	C
4	6	3
4	6	5
4	5	3
4	5	5
5	6	3

{ No Non-trivial FD's }

$$\left\{ \begin{array}{l} \text{Rel } R \text{ with} \\ \text{no non-trivial} \\ \text{FD's} \end{array} \right\} \iff \left\{ \begin{array}{l} \text{CK of } R \text{ is} \\ \text{Combination of} \\ \text{all attributes} \end{array} \right\}$$

③ $R(\underline{A}\underline{BC}\underline{D}\underline{E}\underline{F}) \quad \{D \rightarrow \underline{A}, F \rightarrow \underline{E}, C \rightarrow \underline{D}\}$

Soln
 $\Rightarrow (BCF)^+ = \{BCFEDA\}$

BCF : Candidate Key //

④ $R(\underline{A}\underline{B}\underline{C}\underline{D}\underline{E}\underline{F}) \quad \{E \rightarrow \underline{B}, F \rightarrow \underline{C}, A \rightarrow \underline{F}\}$

Soln
 $\Rightarrow (ADE)^+ = \{ADEFCB\}$

ADE : Candidate Key //

NOTE:-

① If some attribute (x) present in relational schema (R) but not present in given non-trivial FD set Then x must be part of every candidate key of R.

② If some attribute x is belongs to determinant of non-trivial FD but not determiner of any non-trivial FD then x must be part of every candidate Key of R.

⑤ R (A B C D) { AB → C, C → D, D → B }

Soln
Step 1: find one CK

(AB) ⁺ = {ABCD}	<u>A⁺ = A</u> , <u>B⁺ = B</u>	
(AD) ⁺ = {ABCD}	<u>D⁺ = DB</u>	A B ^D
(AC) ⁺ = {ABCD}	<u>C⁺ = CDB</u>	A D C A D B

Candidate Keys : { AB, AD, AC } 3 CK's in R //

Non Trivial FD
X → Y Prime Attribute : At least two CK's in R

⑥ R (A B C D) { A → B, B → C, C → A }

Soln

AD⁺ = {ABCD}

$$AD^+ = \{ABCD\}$$

$$CD^+ = \{ABCD\}$$

$$BD^+ = \{ABCD\}$$

$\Rightarrow$ Candidate Keys: $\{\underline{AD}, \underline{CD}, \underline{BD}\}$

⑦ $R(ABCD)$ $\{AB \rightarrow \overset{\text{Prime}}{CD}, C \rightarrow \overset{\text{Prime}}{A}, D \rightarrow \overset{\text{Prime}}{B}\}$

Soln

$$AB \rightarrow C$$

$$AB \rightarrow D$$

$$(AB)^+ = \{ABCD\} \quad A^+ = A, B^+ = B$$

$$(AD)^+ = \{ABCD\} \quad A^+ = A, D^+ = DB$$

$$(CD)^+ = \{ABCD\} \quad C^+ = CA$$

$$(BC)^+ = \{ABCD\}$$

$$\{\underline{AB}, \underline{AD}, \underline{CD}, \underline{BC}\} \quad 4 \text{ CK'S} //$$

⑧ $R(ABCDEF)$ $\{AB \rightarrow C, C \xrightarrow{D} DE, E \rightarrow F, F \rightarrow A\}$
 $\uparrow$
 $C \rightarrow D, C \rightarrow E$

Soln

$$\{\underline{AB}, \underline{BF}, \underline{BE}, \underline{BC}\} : \text{Candidate Keys} //$$

⑨ $R(ABCDEF)$ $\{AB \rightarrow \overset{\text{Prime}}{C}, C \xrightarrow{A \oplus E \times} \overset{\text{Prime}}{D}, CD \rightarrow \overset{\text{Prime}}{EA}, DE \rightarrow \overset{\text{Prime}}{F}, EF \rightarrow \overset{\text{Prime}}{B}\}$

Soln

$$(AB)^+ = \{ABCDEF\} \quad \underline{A^+ = A}, \underline{B^+ = B}$$

$$(AEF)^+ = \{ABCDEF\} \quad EF^+ = EFB$$

$$(ADE)^+ = \{ABCDEF\} \quad AD^+ = AD, AE^+ = AE, DE^+ = DEFB$$

$$(\cancel{CD})^+ = \{ABCDEF\} \quad C^+ = \{CDEAFB\}$$

$$\{\underline{AB}, \underline{AEF}, \underline{ADE}, \underline{C}\} \text{ Candidate Keys.} //$$

⑩ $R(ABCDEFGG) \{ AB \rightarrow CD, AF \rightarrow D, DE \rightarrow F, C \rightarrow G, F \rightarrow E, G \rightarrow A \}$

Soln

6 Candidate Keys

* Properties of Decomposition :-

- 1) Loss less join decomposition.
- 2) Dependency Preserving decomposition.

$\Rightarrow$ Loss less join decomposition :-

- Relational Schema R with instance r decomposed into sub relations $R_1 R_2 R_3 \dots R_n$

$\bowtie$: Natural Join

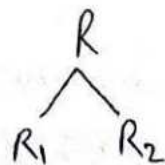
* In General

$$\{ R_1 \bowtie R_2 \bowtie \dots \bowtie R_n \} \supseteq R$$

* If $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n = R$ then
loss less join decomposition

* If $R_1 \bowtie R_2 \bowtie \dots \bowtie R_n \supset R$ then
lossy join decomposition.

$\Rightarrow$



- ① $R_1 \bowtie R_2 = R$:- Loss less join decomposition
- ② $R_1 \bowtie R_2 \supset R$:- Lossy decomposition
- ③ $R_1 \bowtie R_2 \subset R$:- Not possible

Eg:-

R

Sid	Sname	Cid
S1	A	C1
S1	A	C2
S2	B	C1
S3	B	C2
S4	C	C2

$\{ \text{Sid} \rightarrow \text{Sname} \}$

SidCid : Candidate Key

① Decompose R into sub relations $R_1(\text{Sid Sname})$ $R_2(\text{Sid Cid})$

R_1

Sid	Sname
S1	A
S2	B
S3	B
S4	C

R_2

Sid	Cid
S1	C1
S1	C2
S2	C1
S3	C2
S4	C2

$\text{Sid} \rightarrow \text{Sname}$

Lossless Join Decomposition

$(R_1 \bowtie R_2) = R$

Sid	Sname	Cid
S1	A	C1
S1	A	C2
S2	B	C1
S3	B	C2
S4	C	C2

② Decompose R into sub relations $R_1(\text{Sid Sname})$ $R_2(\text{Sname Cid})$

R_1

Sid	Sname
S1	A
S2	B
S3	B
S4	C

R_2

Sname	Cid
A	C1
A	C2
B	C1
B	C2
C	C2

$\text{Sid} \rightarrow \text{Sname}$

Lossy Join Decomposition

$(R_1 \bowtie R_2) \supset R$

Sid	Sname	Cid
S1	A	C1
S1	A	C2
S2	B	C1
S2	B	C2
S3	B	C1
S3	B	C2
S4	C	C2

Spurious tuples

NOTE:- ①

⇒ Relational Schema R with FD set F decomposed into sub relations R_1, R_2

Given decomposition is 'loss less join' iff

① $R_1 \cup R_2 = R$ // every attribute of R must be in sub relations
and

② $(R_1 \cap R_2) \rightarrow R_1$ $(R_1 \cap R_2)^+ = \{ \text{all attributes of } R_1 \}$
(OR) SK for R_1

$(R_1 \cap R_2) \rightarrow R_2$ $(R_1 \cap R_2) : \text{SK for } R_2$

Q) $R(ABCDE) \{ AB \rightarrow C, C \rightarrow D, B \rightarrow E \}$

Test given decomposition is LLJ or not?

i) $\{ ABC, CD \}$

$$\left\{ \begin{array}{l} R_1 \cup R_2 = R \\ \text{and} \\ (R_1 \cap R_2)^+ = R_1/R_2 \end{array} \right\}$$

Ans

$$R_1 \cup R_2 \neq R$$

Attribute E lost

becoz of decomposition

⇒ Lossy Join decomp...

ii) $\{ ABC, DE \}$

Ans

$$R_1 \cup R_2 = R \checkmark$$

$$R_1 \cap R_2 = \emptyset$$

⇒ Lossy Join decomp...

iii) $\{ ABCD, CDE \}$

Ans

$$R_1 \cup R_2 = R \checkmark$$

$$(R_1 \cap R_2)^+ = \{ CD \}$$

⋮
Not SK for R_1, R_2

⇒ Lossy Join decomp...

$$iv) \{ \overset{R_1}{ABCD}, \overset{R_2}{BE} \}$$

Ans $R_1 \cup R_2 = R$

$$(R_1 \cap R_2)^+ = (B)^+ = \{BE\}$$

$\uparrow$
SK for

$\rightarrow$ Loss less join decomp.

$$v) \{ \overset{R_1}{ABC}, \overset{R_2}{ABDE} \}$$

Ans $R_1 \cup R_2 = R$

$$(R_1 \cap R_2)^+ = (AB)^+ = ABCDE$$

$\uparrow$
SK for both Rel R_1, R_2

$\rightarrow$ Loss less Join decomp

NOTE :- (2)

$\Rightarrow$ R with FD set F decomposed into $R_1 R_2 R_3 \dots R_n$

i) $(R_1 \cup R_2 \cup R_3 \dots \cup R_n) = R$

(and)

ii) If $(R_i \cap R_j)$ is SK for R_i or R_j then Join R_i, R_j

If all rel can join into single Rel then LLJ.

Q) $R(ABCDEFGH) \{ AB \rightarrow C, BC \rightarrow A, B \rightarrow D, D \rightarrow E, E \rightarrow FG \}$

i) $\{ \overset{R_1}{ABC}, \overset{R_2}{ADE}, \overset{R_3}{EFG} \}$

Soln $\Rightarrow R_1 \cup R_2 \cup R_3 = R \checkmark$
and

$\Rightarrow R_1(ABC) \quad R_2(ADE) \quad R_3(EFG)$

$\downarrow$ $\swarrow$ $\bowtie$ $\searrow$ E: SK of R_3
 $R_1(ABC)$ $R_{23}(ADEFG)$

$A^+ = \{A\}$

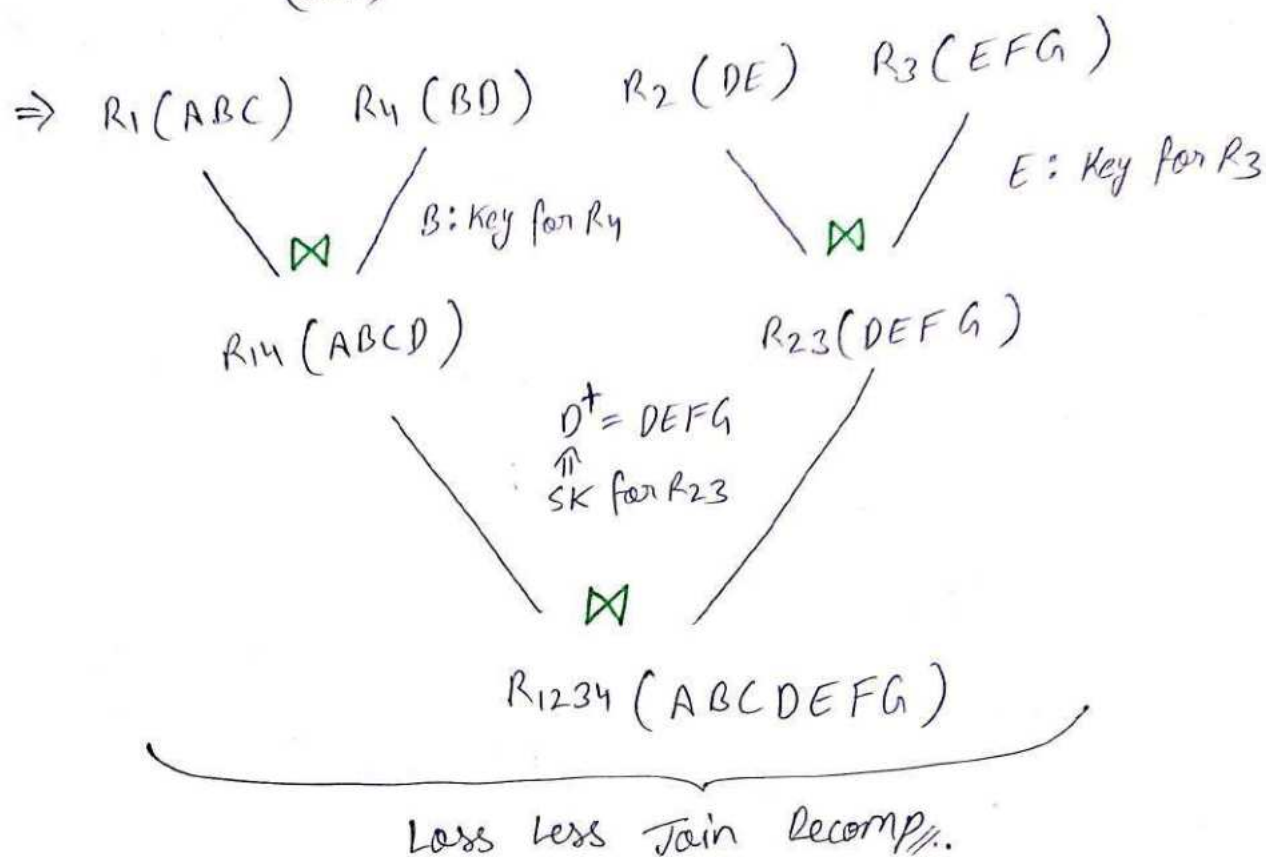
$R_2 \cap R_3$
 $\Rightarrow E^+ = EFG$

Lossy Join Decomp... //

$$ii) \{ \overset{R_1}{ABC}, \overset{R_2}{DE}, \overset{R_3}{EFG}, \overset{R_4}{BD} \}$$

Soln $\Rightarrow R_1 \cup R_2 \cup R_3 \cup R_4 = R$

(and)



$\Rightarrow$ Dependency Preserving Decomposition :-

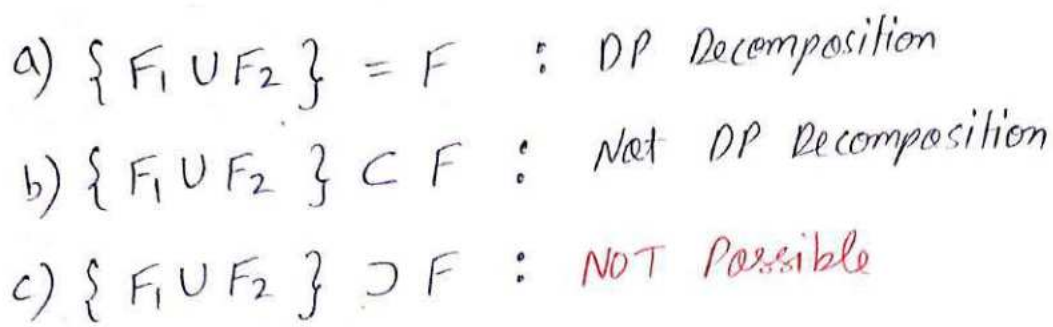
- Relational Schema R with FD set F decomposed into sub relations $R_1 R_2 R_3 \dots R_n$ with FD sets $F_1 F_2 F_3 \dots F_n$ respectively.

* In General

$$\{F_1 \cup F_2 \cup F_3 \dots \cup F_n\} \subseteq F$$

* If $\{F_1 \cup F_2 \cup F_3 \dots \cup F_n\} = F$ then dependency preserving decomposition.

* If $\{F_1 \cup F_2 \cup F_3 \dots \cup F_n\} \subset F$ then Not Dependency preserving decomposition.



$\Rightarrow R(ABCDE) \{ A \rightarrow BC, B \rightarrow D, CD \rightarrow E, E \rightarrow A \}$
what are non-trivial FD's of Sub Rel $R_1(ABCE)$

$$\begin{aligned} A^+ &= \{A \underline{BCD} \underline{E}\} \\ B^+ &= \{B \underline{D}\} \\ C^+ &= \{C\} \\ E^+ &= \{E \underline{A} \underline{BCD}\} \\ BC^+ &= \{BC \underline{DEA}\} \end{aligned}$$

Q) $R(ABCDE)$ $\xrightarrow{F=}$ $\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow BE\}$ decomposed into $R_1(AB) R_2(BC) R_3(CD) R_4(DE)$ test dependency preserving?

Soln

Non-Trivial FD's sub relations

$R_1(AB)$	$R_2(BC)$	$R_3(CD)$	$R_4(DE)$
$A \rightarrow B$	$B \rightarrow C$	$C \rightarrow D$	$D \rightarrow E$
F_1	F_2	F_3	F_4

To find FD's of sub Rel :-

$A \rightarrow B$
 $B \rightarrow C$
 $C \rightarrow D$
 $D \rightarrow E$

To find FD's of sub Rel :-

$$A^+ = \{A B C D E\}$$
$$B^+ = \{ B, \underline{C}, D, E \}$$
$$C^+ = \{C, D, B, E\}$$
$$D^+ = \{ D \underline{B} \underline{E} C \}$$

$\Rightarrow$ DP Recomp. //

$$(F_1 U F_2 U F_3 U F_4) = F_{//}$$

Q) $R(ABCD)$ $\{AB \rightarrow CD, D \rightarrow A\}$ decomposes in $\{ABC, BCD, AD\}$

Soln

	$R_1(ABC)$	$R_2(BCD)$	$R_3(AD)$
Non-trivial FD's of Sub-rel	$AB \rightarrow C$	$BD \rightarrow C$	$D \rightarrow A$
	F_1	F_2	F_3

$D^+ = DA$
 $CD^+ = CDA$
 $BD^+ = BDAC$
 $B^+ = B$
 $C^+ = C$
 $BC^+ = BC$

$\Rightarrow \{F_1 \cup F_2 \cup F_3\} \subset F$
 Not DP decomp //

$\underline{F}$
 $AB \rightarrow C \checkmark$
 $AB \rightarrow D \times$
 $D \rightarrow A \checkmark$

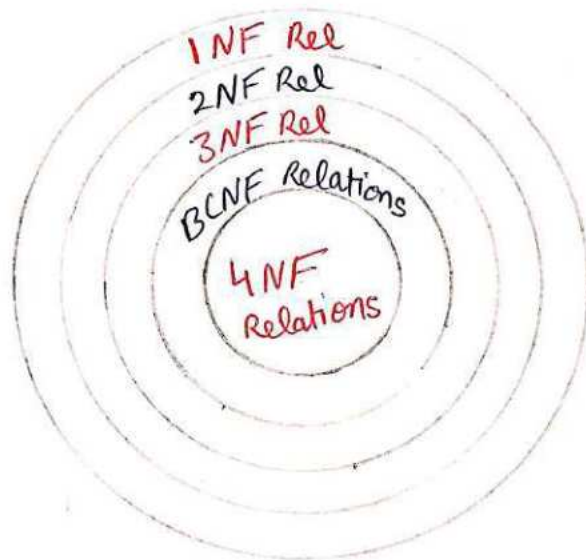
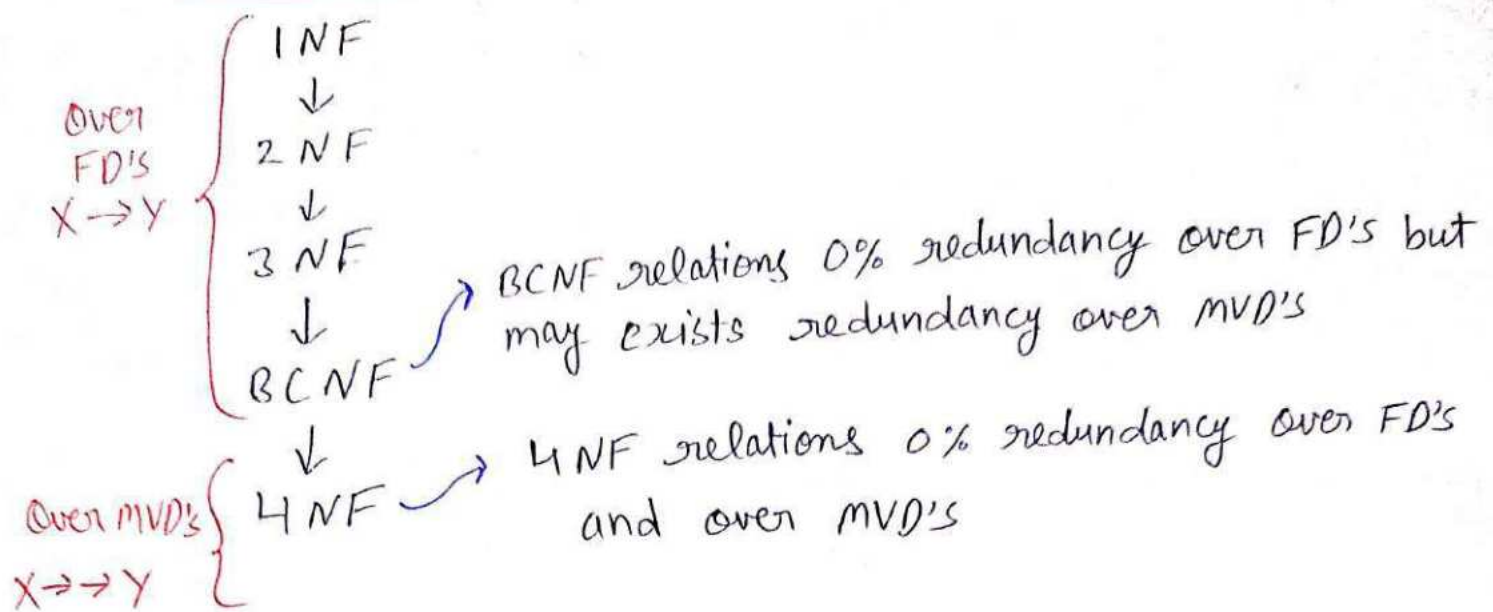
* Normal Forms :-

$\Rightarrow$ Used to identify degree of redundancy & used to reduce/eliminate redundancy.

Redundancy in Rel R

- Over non-trivial FD's
 $X \rightarrow Y$
(Single valued functional dependency)
- To eliminate redundancy over FD's relation should decompose into BCNF
- Over non-trivial MVD's
 $X \twoheadrightarrow Y$
(Multi valued functional dependency)
- To eliminate redundancy over MVD's relation should decompose into 4NF.

⇒ Normal Forms :-



* First Normal Form (1NF) :-

⇒ Default NF of RDBMS Relations

⇒ Relation R is in 1NF iff

No multivalued attributes in R

[Every Attribute of R must be single valued or atomic]

RDBMS
Guideline

Eg: ①

Emp

Eid	Ename	phno
e1	A	p1/p2
e2	B	p2/p3
e3	B	p4

multi valued
Attr. Emp

⇒
(INF Design)

Eid	Ename	phno
e1	A	p1
e1	A	p2
e2	B	p2
e2	B	p3
e3	B	p4

{eid → ename}

{INF design} ⇔ {design CK's using FD set}

Card Key: [Eid phno]

⇒ No multivalued Attr.

⇒ Rel in INF

②

R

Sid	Sname	age	Cid	phno	email
S1	A	20	c1/c2	p1/p2/p3	e1/e2/e3/e4
S2	B	22	c2/c3	p3/p4	e4/e5

2 * 3 * 4

R not in INF {Sid → Sname age}

⇓ INF Design

Sid	Sname	age	cid	phno	email
S1	A	20	c1	p1	e1
⋮	⋮	⋮	⋮	⋮	⋮
S1	A	20	c2	p3	e4
S2	B	22	c2	p3	e4
⋮	⋮	⋮	⋮	⋮	⋮
S2	B	22	c3	p4	e5

24 tuples
for S1

8 tuples
for S2

⇒ {Sid → Sname age}

Card Key: Sid cid phno email

No multivalued Attributes [R in INF]

⇒ Disadvantage of INF Design :- more degree of concurrency.

NOTE:- ① $X \rightarrow Y$ FD of Rel R forms redundancy iff

- 1) Non Trivial FD
and
- 2) X is not superkey.

Not SK
 $X \rightarrow Y$
Non trivial
FD

② $X \rightarrow Y$ FD of Rel R not forms redundancy iff

- 1) Trivial FD [$X \supseteq Y$]
- OR
- 2) X is superkey

Non
Trivial
FD

$X \rightarrow Y : SK$

Eg:- ① R

Sid	Sname	Cid
S1	A	C1
S1	A	C2
S1	A	C3
S2	B	C1
S3	B	C2
S4	C	C2

Redundancy

$\{ \text{Sid} \rightarrow \text{Sname} \}$
not SK
forms Redundancy

SidCid : Cand Key

② R

eid	Ename	dob
e1	A	2000
e2	A	2000
e3	A	2002
e4	B	1998
e5	B	1999

$\{ \text{eid} \rightarrow \text{ename dob} \}$
SK
eid : Cand Key
(No Redundancy)

③

eid	ename	rating	Hourly wage
e1	A	6	5000
e2	A	6	5000
e3	B	7	6000
e4	C	7	6000
e5	B	7	6000
e6	C	8	7000

$\{ \text{eid} \rightarrow \text{ename rating} \}$
SK
 $\text{rating} \rightarrow \text{Hourly wage}$
not SK
Forms Redundancy

eid : Cand Key

④ $R(A B C D E F)$

$\{ \overset{SK}{AB} \rightarrow C$

① $B \rightarrow D$

② $D \rightarrow E$

③ $BE \rightarrow F$

④ $C \rightarrow A$

}

Forms
Redundancy

How many given non trivial FD's forms redundancy?

Cand Key : $[\underline{AB}, \underline{BC}]$

Prime Attrs : $\{ A, B, C \}$

Non Prime Attrs : $\{ D, E, F \}$

⇒ Non Trivial FD's which forms redundancy :-

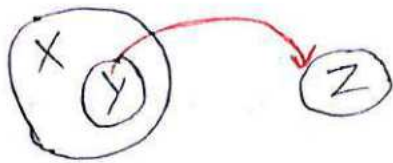
0% Redundancy
over FD's

	Forms Redundancy	$X \rightarrow Y$ non trivial FD with X not SK	1NF	2NF	3NF	BCNF	4NF
			Allowed ✓	X not Allowed	X	X	X
① PD		X: Proper Subset of Cand. Key $\rightarrow$ Y: Non Prime	✓	X not Allowed	X	X	X
②		X: Non Prime Attributes $\rightarrow$ Y: Other Non Prime Attr's	✓	✓	X	X	X
③		X: Proper subset of CK + Non Prime Attr's $\rightarrow$ Y: Non Prime Attr's	✓	✓	X	X	X
④		X: Proper sub Set of CK $\rightarrow$ Y: proper sub Set of other CK	✓	✓	✓	X	X

* Second Normal Form (2NF) :-

⇒ Relational Schema R is in 2NF iff No Partial Dependency in Relation R.

Partial Dependency :-



$Y \rightarrow Z$: Partial Dependency

X: Any Candidate Key of R

$Y \subset X$

Y proper subset of cand. Key

Z: Non Prime Attr. of R

NOTE:- 2NF not allowed redundancy becoz of partial dependency but not eliminates redundancy.

* Third Normal Form (3NF) :-

⇒ Relational Schema R is in 3NF iff every non-trivial FD $X \rightarrow Y$ in R with

① X must be super key of R

or

② Y must be prime attribute of R

Non-Trivial FD

$\underbrace{X} \rightarrow \underbrace{Y}$
SK or Prime

NOTE:-

* 3NF reduces redundancy

* $\left\{ \begin{array}{l} \text{Proper subset of} \\ \text{candidate Key} \end{array} \right\} \longrightarrow \left\{ \begin{array}{l} \text{Proper subset of other} \\ \text{candidate Key} \end{array} \right\}$

allowed in 3NF which is causes redundancy

* BCNF (Boyce - Codd Normal Form) :-

⇒ Relational Schema R is in BCNF iff Every Non-trivial FD $X \rightarrow Y$ in R with X must be Super Key.

for all
Non-Trivial FD's
 $X \rightarrow Y$
SK

• BCNF allowed FD $X \rightarrow Y$ iff

1) $X \rightarrow Y$ is Trivial FD ($X \supseteq Y$)

OR

2) $X \rightarrow Y$ non-trivial FD with X is Super Key.

• BCNF Not allowed FD $X \rightarrow Y$ iff

$X \rightarrow Y$ non trivial FD and X not Super Key.

Q) what is highest Normal form satisfied for the given relational schema $R(A B C D E)$ { $ABD \rightarrow C$, $BC \rightarrow D$, $CD \rightarrow E$ }

Soln
a) 1NF

b) 2NF

c) 3NF

d) BCNF

Cand Keys: $\underline{ABD}$
 $\underline{ABC}$

Soln { $\overbrace{ABD} \rightarrow C$, $BC \rightarrow \overset{\text{Prime}}{D}$, $CD \rightarrow E$ }

① BCNF

✓

X

X

② 3NF

✓

✓

X

$\underbrace{X \rightarrow Y}_{\text{SK OR Prime}}$

③ 2NF

[No PD]

Not PD

Not PD

Not PD

$BC \rightarrow D, CD \rightarrow E \Rightarrow \underline{BC \rightarrow E}$
PD

2NF Testing :-

$$\left\{ \begin{array}{l} \text{Proper subset} \\ \text{of CK} \end{array} \right\}^+ = \left\{ \begin{array}{l} \text{Only Prime} \\ \text{Attributes} \end{array} \right\}$$

$$"ABD" \Rightarrow \begin{cases} A^+ = A \\ B^+ = B \\ D^+ = D \\ AB^+ = AB \\ AD^+ = AD \\ BD^+ = BD \end{cases}$$

$$ABC \Rightarrow \begin{cases} C^+ = C \\ BC^+ = BCD \text{ (Non Prime)} \end{cases}$$

$BC \rightarrow E$: PD
R not in 2NF //

Q) Find highest NF of given Relational Schemas?

① $R(ABCD) \{AB \rightarrow C, BC \rightarrow D\}$

Soln

$$CK \Rightarrow \underline{AB} \quad \begin{matrix} \checkmark & X \Rightarrow \text{not BCNF} \\ \checkmark & X \Rightarrow \text{not 3NF} \end{matrix}$$

$$2NF_{\text{Test}} : AB \Rightarrow A^+ = A, B^+ = B \quad \text{No PD} \Rightarrow 2NF_{\text{ans}}$$

② $R(\underline{ABC} \text{ (Prime)} D) \{AB \rightarrow C, C \xrightarrow{\text{Prime}} A, AC \rightarrow D\} \quad CK = \{\underline{A}, \underline{BC}\}$

Soln

$$\begin{matrix} \checkmark & X & X & \Rightarrow \text{not BCNF} \\ \checkmark & \checkmark & X & \Rightarrow \text{not 3NF} \end{matrix}$$

$$2NF_{\text{Test}} : \underline{AB} \Rightarrow \begin{matrix} A^+ = A \\ B^+ = B \end{matrix}$$

$$\underline{BC} \Rightarrow C^+ = CA \text{ (Non Prime)}$$

$\{C \rightarrow D\}$ PD $\Rightarrow$ Not in 2NF

Ans = 1NF //

③ $R(ABCD) \{A \rightarrow B, B \rightarrow AC, C \rightarrow D\}$ $CK \Rightarrow \{ \underline{A}, \underline{B} \}$ simple CK's
Soln
 $\checkmark \quad \checkmark \quad \times \Rightarrow$ Not BCNF
 $\checkmark \quad \checkmark \quad \times \Rightarrow$ Not 3NF

Because of simple CK's [Proper subset of CK] $\rightarrow$ [Non Prime] } = 2NF
 Partial Dependency not possible // Ans

④ $R(ABCDEF)$ $\{AB \rightarrow C, C \rightarrow DE, E \rightarrow F, F \rightarrow A\}$ *non prime*

Soln $C \rightarrow D$: Partial Dependency (not in 2NF)

CK's
 $\downarrow$
 $\{ \underline{AB}, \underline{BF}, \underline{BE}, \underline{BC} \}$

Ans = 1NF //

⑤ $R(\underline{ABCDEF}) \{AB \rightarrow C, C \rightarrow D, CD \rightarrow EA, DE \rightarrow F, EF \rightarrow B\}$
all attributes are prime
 $\checkmark \quad \checkmark \quad \checkmark \quad \times \quad \times \Rightarrow$ not BCNF
 $\checkmark \quad \checkmark \quad \checkmark \quad \checkmark \quad \checkmark \Rightarrow$ 3NF // Ans

$CK \Rightarrow \{ \underline{AB}, \underline{AEF}, \underline{ADE}, \underline{C} \}$

⑥ $R(\underline{ABCDE}) \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
Prime Attributes
 $\times \quad \times \quad \times \quad \times \Rightarrow$ not BCNF
 $\checkmark \quad \checkmark \quad \checkmark \quad \checkmark \Rightarrow$ 3NF // Ans

NOTE :-

① If Relation R with only simple candidate Keys then R always in 2NF but may not in 3NF.

② Relation R with only prime attributes always in 3NF but may not in BCNF.

③ Relation R with No non trivial FD's is always in BCNF may not in 4NF.

$R(ABC)$ no non trivial FD's

Rel R with no non trivial FD $\Rightarrow \left\{ \begin{array}{l} \text{no redundancy in} \\ R \text{ over FD's} \end{array} \right\} [R \text{ in BCNF}]$

④ If Relation R with only two attributes then R always in BCNF [also in 4NF, 5NF, 6NF].
 $\downarrow$ JDNF $\rightarrow$ DKNF

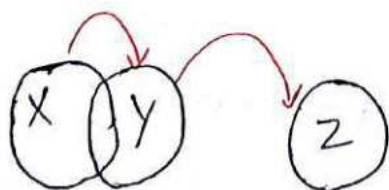
$R(AB)$ a) $\{A \rightarrow B\}$ b) $\{B \rightarrow A\}$ c) $\{A \rightarrow B, B \rightarrow A\}$
A BCNF B BCNF A B BCNF

d) $\{\text{No non trivial FD's}\}$
AB BCNF

$\Rightarrow$ 3NF: Relation R in 3NF iff no transitive dependency in R .

* Transitive Dependency:-

Non Prime Attribute (Z) Transitivity determined by super key (X) $\Rightarrow$ not allowed in 3NF



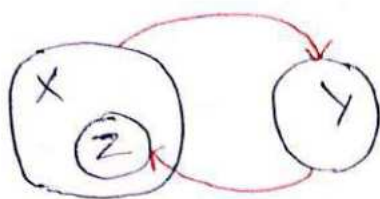
X : any CK of R

Y : not SK

Z : Non Prime

$\{ X \rightarrow Y, Y \rightarrow Z \}$
 $\uparrow$ SK $\uparrow$ Non Prime

- * Prime Attribute (Z) transitively determined by superkey (X) is allowed in 3NF but not allowed in BCNF.



X : CK

Y : Not SK

Z : Prime Attr

$\{ \underset{\substack{\uparrow \\ \text{SK}}}{X} \rightarrow Y, Y \rightarrow \underset{\substack{\uparrow \\ \text{Prime}}}{Z} \}$
 $\underset{\substack{\uparrow \\ \text{not SK}}}{Y}$

allowed in 3NF but not allowed in BCNF.

* Decompose Relation into Higher Normal Forms :-

⇒ Decompose relation into 2NF, 3NF, BCNF with lossless Join decomposition & Dependency Preserving decomposition.

* Decompose into 2NF :-

$R(\dots XY) \{ \dots \underbrace{X \rightarrow Y}_{\text{Assume Partial Dependency}} \} // R \text{ not in 2NF}$

$x^+ = \{XY\}$

$R_1(\underline{X}Y) \quad R_2(\dots \overset{FK}{\cancel{X}}) \quad \left\{ \begin{array}{l} LLJ \checkmark \\ DP \checkmark \\ NO PD \checkmark \\ [2NF] // \end{array} \right.$

$\{X \rightarrow Y\}$

Q) Decompose into higher NF

R

Sid	Sname	Cid
S1	A	C1
S1	A	C2
S1	A	C3
S2	B	C2
S3	B	C3

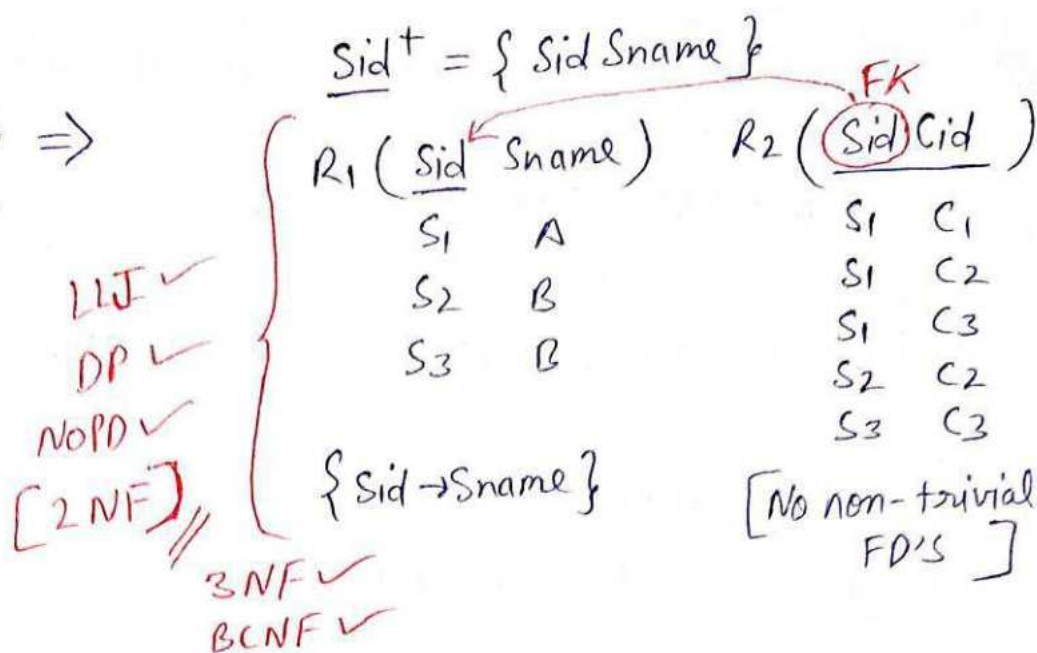
$\{ \text{sid} \rightarrow \text{Sname} \}$

Sid Cid : Cand Key

R not in 2NF

Soln

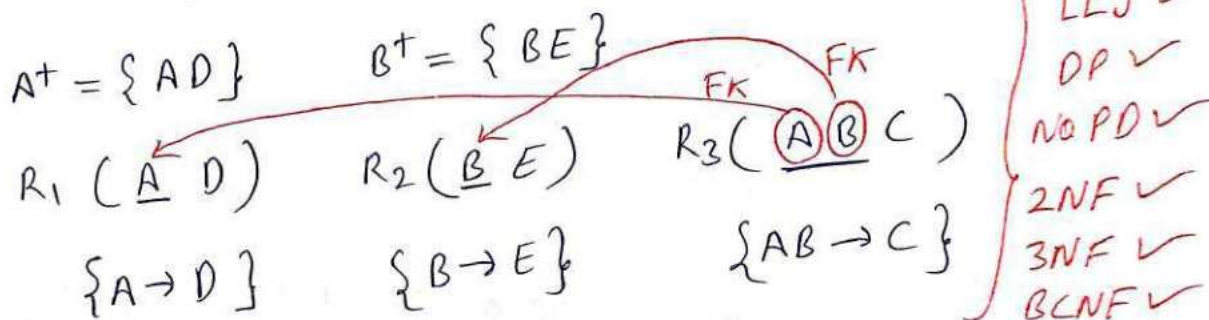
2NF Decomp $\Rightarrow$



Q) $R(ABCDE) \{ AB \rightarrow C, \underline{A \rightarrow D}, \underline{B \rightarrow E} \}$ R not in 2NF
decompose into 2NF:-

Soln

AB : Cand Key

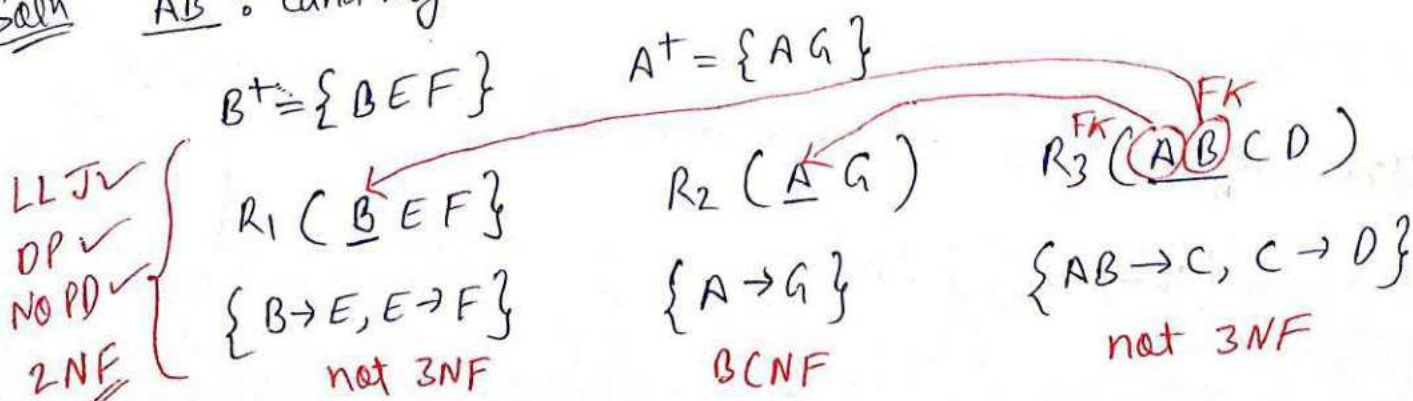


$\Rightarrow$ min 3 relations & 2 FK's //

Q) $R(ABCDEFG) \{ AB \rightarrow C, C \rightarrow D, \underline{B \rightarrow E}, E \rightarrow F, \underline{A \rightarrow G} \}$
 R not in 2NF
decompose into 2NF :-

$\{ \text{min 3 relations \& 2 FK's} \}$

Soln AB : Cand Key



3NF Design $\Rightarrow$ $R_1(\underline{B}, E)$ $R_4(\underline{E}, F)$ $R_2(\underline{A}, G)$ $R_3(\underline{A}, B, C)$ $R_5(\underline{C}, D)$

LLT ✓
DP ✓
3NF ✓
BCNF ✓

$\{B \rightarrow E\}$ $\{E \rightarrow F\}$ $\{A \rightarrow G\}$ $\{AB \rightarrow C\}$ $\{C \rightarrow D\}$

$\Rightarrow$ min 5 Relations & 4 FK'S.

Q) How many minimum relations required to satisfy 2NF for given relation $R(ABCD)$ $\{\underbrace{A \rightarrow B}_{PD}, \underbrace{C \rightarrow D}_{PD}\}$.

Soln AC : Cand Key R not in 2NF

2NF decomp :- $A^+ = \{AB\}$ $C^+ = \{CD\}$

LLT ✓
DP ✓
NO PD ✓
2NF ✓
3NF ✓
BCNF ✓

$R_1(\underline{A}, B)$ $R_2(\underline{C}, D)$ $R_3(\underline{AC})$

$A \rightarrow B$ $C \rightarrow D$ No Non trivial FD

$\Rightarrow$ min 3 relations & 2 FK'S

Q) How many minimum relations required to satisfy 2NF for given Relations $R(ABCDEF)$ $\{A \rightarrow D, B \rightarrow E, C \rightarrow F\}$

Ans $\Rightarrow$ 4 Relations & 3 FK'S

Q) $R(ABCDEFGH)$ $\{AB \rightarrow C, C \rightarrow D, A \rightarrow E, E \rightarrow F, B \rightarrow G, G \rightarrow H\}$

i) How many min Rel required for 2NF decomposition?

Ans $\Rightarrow$ 3 tables & 2 FK'S

ii) How many min Rel required for 3NF decomposition?

Ans $\Rightarrow$ 6 tables & 5 FK'S

Q) $R(ABCDE)$ $\{AB \rightarrow C, BC \rightarrow A, AC \rightarrow B\}$ decompose into BCNF?

Soln $CK \rightarrow \{ \underline{ABDE}, \underline{BCDE}, \underline{ACDE} \}$ R in 3NF but not in BCNF

Decompose into BCNF :-

$R_1(\underline{ABC})$
 $\{ AB \rightarrow C, BC \rightarrow A, AC \rightarrow B \}$

$CK \rightarrow \{ \underline{AB}, \underline{BC}, \underline{AC} \}$

$R_2(\overset{FK}{\underline{AB}}DE)$
 No non trivial FD's

LLJ ✓
 DP ✓
 BCNF ✓

$\Rightarrow$ min 2 Rel & 1 FK //

Q) $R(ABC)$ $\{AB \rightarrow C, C \rightarrow A\}$ decompose into BCNF?

R in 3NF but not in BCNF

Soln $\{ \underline{AB}, \underline{BC} \} : \text{Cand Key}$

BCNF :
 Decomp

$R_1(\underline{AC})$
 $C \rightarrow A$

$R_2(\underline{BC})$
 No non trivial FD

LLJ ✓
 BCNF ✓
 Not DP ✓

$\Rightarrow$ Not possible to decompose with DP & BCNF

Q) $R(ABCD)$ $\{ABC \rightarrow D, D \rightarrow A\}$ decompose into BCNF?

R in 3NF but not in BCNF

Soln $[\underline{ABC}, \underline{BCD}] : \text{Cand Key}$

BCNF :-
 Decomp

$R_1(\underline{DA})$
 $D \rightarrow A$

$R_2(\underline{BCD})$
 No non trivial FD's

LLJ ✓
 BCNF ✓
 Not DP ✓

$\Rightarrow$ Not possible to decompose with DP & BCNF.

⇒ DB Design goals based on Schema Refinement	1NF	2NF	3NF	BCNF	4NF
***① Lossless Join Decomposition	Possible for every Rel decomposition	Possible for every Rel decomposition	Possible for every Rel Decomp.	Possible for every Rel Decomp.	may not possible for every Decomp.
***② Dependency Preserving Decomposition	Possible for every Rel Decomp	Possible for every Rel Decomp	Possible for every Rel Decomp	may not possible for every Decomp	may not possible for every Decomp
*③ 0% Redundancy	NO	NO	NO	Yes over FD's NO over MVD's	Yes over FD's & MVD's

↑
most Accurate
NF : 3NF

* 5NF [Join Dependency NF] [JDNF] :-

⇒ Decomposition of Rel R into Sub Rel $R_1 R_2 \dots R_n$ is

① 4NF Decomposition and

② $\bowtie (R_1 R_2 R_3 \dots R_n) = R$
[Lossless Join Decomp]

* 6NF [Domain Key NF] [DKNF] :-

⇒ Decomposition of R into $R_1 R_2 \dots R_n$ is in 6NF iff

① Decomposition must be 5NF and

② Every FD, MVD of R must be preserved in Sub Relation
[DP Decomp],

Queries

⇒

Query Language

- Procedural Query Language
Formulation of How data retrieved (and) what data retrieved from DB table

(Relational Algebra)

- Non Procedural Query Language
Formulation of what data retrieved from DB tables.

(Relational Calculus)

[Uses First order logic & Predicate Calculus formulas]

Tuple Relational Calculus [TRC]

[SQL close to TRC]

Domain Relational Calculus [DRC]

[Query By Example (QBE) close to DRC]

* Relational Algebra :-

⇒ Basic Operators :-

π : Projection
 σ : Selection
 ρ : Rename

} unary operators

$\cup$: Union
 $-$: Set Difference (minus)
 $\times$: Cross Product

} Binary Operators

⇒ Derived Operators :-

$\cap$: Intersection [using $-$]

$\bowtie$: Join [using $\times, \sigma, \pi$]

$(/ \text{ or } \div)$: Division [using $\pi, \times, -$]

* Projection (π) :- Used to project required attributes.

$\pi(R)$
attribute-list

* Selection (σ) :- Used to select records based on selection condition.

$\sigma(R)$
condition

Eg:-

R

A	B	C
✓ 5	6	5
✓ 4	6	5
✗ 7	2	8
✓ 2	2	7
✓ 2	2	7

bag

$\pi_{BC}(R) \Rightarrow$

B	C
6	5
2	8
2	7

~~$\pi(R)$~~
 ~~$B > 10$~~

$\sigma(R) =$
 $A \leq 5$

A	B	C
5	6	5
4	6	5
2	2	7

Distinct tuples

Result of RA Query
always distinct records

⇒ $\{2, 3, 4, 5\}$: Set
 $\{2, 2, 3, 3, 4, 5\}$: bag } Set theory

* Rename (ρ) :- Used to rename table or attributes for Query processing.

Eg:- Student(sid, sname, age) // Given DB Table

① $\rho(\text{Temp, Student}) \Rightarrow \text{Temp}$

Sid	Sname	age

② $\rho(\text{Student}) \Rightarrow \text{Student}$

I	N	A

I, N, A

③ $\rho(\text{Student}) \Rightarrow \text{Student}$

I	Sname	A

sid $\rightarrow$ I
age $\rightarrow$ A

* Cross Product (X) :- $R \times S$ results all attributes of R followed by all attributes of S and each record of R pairs with every record of S.

Eg:-

pairs with every record of S.

Diagram illustrating the Cartesian product of two relations, R and S, resulting in the relation R x S.

Relation R (Auity: X):

A	B	C
4	2	6
5	2	5
6	3	5

n distinct tuples

Relation S (Auity: Y):

C	D
8	2
5	4

m distinct tuples

Resulting Relation R x S (Auity: X+Y):

A	B	C	C	D
4	2	6	8	2
4	2	6	5	4
5	2	5	8	2
5	2	5	5	4
6	3	5	8	2
6	3	5	5	4

n x *m* tuples
Cardinality

$\Rightarrow$ R with n tuples & S with o tuples then $R \times S$ result: o tuples.

* Join Operators :-

① Natural Join ($\bowtie$) :-

$\Rightarrow$ Cross Product followed by Selection followed by Projection.
 $(\times)$ (σ) (π)

$$R \bowtie S = \pi_{\text{distinct attributes}} \left(\sigma_{\substack{\text{equality b/w common} \\ \text{attributes of R \& S}}} (R \times S) \right)$$

Eg:-

①

R	A	B	C
	4	2	6
	5	2	5
	6	3	5

S	C	D
	8	2
	5	4
	4	6

$$R \bowtie S = \pi_{ABCD} \left(\sigma_{R.C=S.C} (R \times S) \right)$$

$R \bowtie S$

A	B	C	D
5	2	5	4
6	3	5	4

② $R(ABC) \ S(BCDE)$

$$R \bowtie S = \pi_{ABCDE} \left(\sigma_{\substack{R.C=S.C \\ R.B=S.B}} (R \times S) \right)$$

$R \times S$

A	B	C	B	C	D	E

③ $R(AB) \ S(CD)$

$$R \bowtie S = R \times S$$

$\Rightarrow$ Natural Join equal to cross product if Join condition empty (no common attribute)

$\Rightarrow$

R	Sid	Sname	Cid	Cname
	S1	A	C1	DB
	S1	A	C2	OS
	S2	B	C2	OS

Lossy
Join
Decomp.

R_1 (Sid Sname)	R_2 (Cid Cname)
S1 A	C1 DB
S2 B	C2 OS

$$(R_1 \bowtie R_2) = (R_1 \times R_2) \supseteq R$$

causes spurious tuples

② Conditional Join ($\bowtie_c$) :-

$$R \bowtie_c S = \sigma_c(R \times S)$$

Eg:-

R	A	B	C
	4	2	6
	5	2	5
	6	3	5

S	C	D
	8	2
	5	4
	4	6

$$R \bowtie S = \sigma_{R.C > S.C}(R \times S)$$

A	B	C	C	D
4	2	6	5	4
4	2	6	4	6
5	2	5	4	6
6	3	5	4	6

NOTE:-

- 1) Arity of $R \bowtie S \leq$ Arity of $R \times S$
- 2) Arity of $R \bowtie_c S =$ Arity of $R \times S$

③ Outer Joins :-

i) Left Outer Join ($\bowtie_L$) :-

$R \bowtie_L S$: results ($R \bowtie S$) $\cup$ (records of R those are failed join condition with null values for remaining attributes of S).

R	A	B	C
X	4	2	6
✓	5	2	5
✓	6	3	5

=

S	C	D
	8	2
	5	4
	4	6

S	C	D
	8	2
	5	4
	4	6

$R \bowtie_L S$	A	B	C	D
	5	2	5	4
	6	3	5	4
	4	2	6	null

ii) Right Outer Join ($\bowtie_R$) :-

$R \bowtie_R S$	A	B	C	D
	5	2	5	4
	6	3	5	4
	null	null	8	2
	null	null	4	6

iii) Full Outer Join (IX) :-

$$R \text{ IX } S = (R \text{ FDS } S) \cup (R \text{ DIS } S)$$

A	B	C	D
5	2	5	4
6	3	5	4
4	2	6	null
null	null	8	2
null	null	4	6

* Join Queries :-

⇒ Model I :- { some / Any / Atleast one } $R \bowtie_c S / R \bowtie S / \sigma_c(R \times S)$

$R(A...) \times S(B...)$ Retrieve A values of R those are more than some B values of S.

> Some

R	A...	X	S	B...
	10			15
	20			25
	30			35
	40			

① $\pi_{R.A} \left(\sigma_{R.A > S.B} (R \times S) \right)$

(OR)

② $\pi_{R.A} (R \bowtie S)$

① $\text{Emp}(\text{Eid sal dno})$, Retrieve eid's whose salary less than some emp salary of dept 5.

< Some

⇒	Emp(eid sal dno)	Mc	Emp(eid sal dno)
	e1 20 4		e2 30 5
	e2 30 5		e4 50 5
	e3 40 4		
	x e4 50 5		
	x e5 60 3		

$$\textcircled{1} \pi_{eid} \left(\sigma_{sal < 5} \left(Emp \times \rho \left(\sigma_{\substack{Emp \\ dno=5}} \right) \right) \right)$$

(OR)

$$\textcircled{1} \pi_{eid} \left(Emp \bowtie_{sal < 5} \rho \left(\sigma_{\substack{Emp \\ dno=5}} \right) \right)$$

② Stud(sid age gen), Retrieve female students whose age more than age of some male student.

$\Rightarrow$ Stud(sid age gen) Stud(sid age gen)

$\uparrow$ female
 $\uparrow$ male

$\nearrow$ Some

$$\textcircled{1} \pi_{sid} \left(\sigma_{\substack{Stud \\ gen=female}} \bowtie_{age > A} \rho \left(\sigma_{\substack{Stud \\ gen=male}} \right) \right)$$

$\Rightarrow$ Less Cost
[No. of comparisons]

(OR)

$$\textcircled{1} \pi_{sid} \left(\sigma_{\substack{Stud \\ gen=female \\ \wedge A=male \\ \wedge age > A}} \right)$$

$\Rightarrow$ More Cost

$\Rightarrow R(ABC) \ S(CDE)$, Write optimized queries for the following.

$$\textcircled{1} \sigma_{\substack{A > 10 \\ \wedge E < 10 \\ \wedge R.C > S.C}} (R \times S) = \sigma_{\substack{A > 10 \\ R.C > S.C}} \left(\sigma(R) \times \sigma_{E < 10}(S) \right)$$

$$\textcircled{2} \sigma_{\substack{A > 5 \\ \wedge D = 10 \\ \wedge R.C < S.C}} (R \bowtie S) = \sigma_{\substack{A > 5 \\ R.C < S.C}} \left(\sigma(R) \bowtie \sigma_{D=10}(S) \right)$$

$$\textcircled{3} \sigma_{\substack{A > 10 \\ \downarrow \text{natural join}}} (R \bowtie S) = \sigma_{A > 10} (\sigma(R) \bowtie S)$$

 $\Rightarrow$ Model II :- Every / All

$R(A\dots) \ S(B\dots)$ Retrieve A values of R those are more than every B values of S.

$\leq$ Some

R	A...
10	
20	
30	
✓ 40	

R	A...
✓ 10	
✓ 20	
✓ 30	
X 40	

S	B....
	15
	25
	30

$\neg(>\text{every}) \equiv \leq \text{Some}$

$\rightarrow \text{not}$

$$\left\{ \begin{array}{l} \text{A val of R} \\ >\text{every B of S} \end{array} \right\} \equiv \left\{ \begin{array}{l} \text{All A} \\ \text{val of R} \end{array} \right\} - \left\{ \begin{array}{l} \text{A val of R} \\ \leq \text{some B of S} \end{array} \right\}$$

$\pi_A(R) - \pi_A(R \bowtie_{A \leq B} S)$

① $\text{Emp}(\underline{\text{Eid}} \ \text{sal} \ \text{dno})$ Retrieve eid's whose salary less than every emp salary of dept 5.

$$\Rightarrow \left\{ \begin{array}{l} \text{eid's whose sal} \\ < \text{every emp sal} \\ \text{of dno} = 5 \end{array} \right\} \equiv \left\{ \begin{array}{l} \text{all eid's of} \\ \text{emp relation} \end{array} \right\} - \left\{ \begin{array}{l} \text{eid's whose sal} \\ \geq \text{some emp sal} \\ \text{of dno} = 5 \end{array} \right\}$$

$\boxed{e_1} \equiv$

<u>eid</u>
✓ e₁
e₂
e₃
e₄
e₅

$\geq$ Some

Emp	<u>eid</u>	sal	dno
X	e ₁	40	2
✓	e ₂	50	5
✓	e ₃	60	2
✓	e ₄	70	5
✓	e ₅	80	2

Emp	<u>eid</u>	sal	dno
	e ₂	50	5
	e ₄	70	5

$\swarrow$ 5

$$\pi_{\text{eid}}(\text{Emp}) - \pi_{\text{eid}} \left(\text{Emp} \bowtie_{\text{sal} \geq \text{sal}} \rho \left(\sigma_{\text{dno}=5}(\text{Emp}) \right) \right)_{I, S, D}$$

② Stud (sid age gen), Retrieve female students whose age more than age of every male student.

$$\Rightarrow \left\{ \text{sid's of female stud whose age} \right. \\ \left. > \text{every male stud} \right\} \equiv \left\{ \text{sid's of all female stud} \right\} - \left\{ \text{sid's of female stud} \right. \\ \left. \text{whose age} \leq \text{some age of male stud} \right\}$$

<u>sid</u>
S1
S3
S5

Stud (<u>sid</u> age gen)	Stud (<u>sid</u> age gen)
✓ S1 20 fem	S2 21 male
✓ S3 22 fem	S4 23 male
✗ S5 24 fem	

$$\cdot \pi_{\text{sid}} \left(\sigma(\text{stud})_{\text{gen=fem}} \right) - \pi_{\text{sid}} \left(\sigma(\text{stud})_{\text{gen=fem, age} \leq A} \bowtie \rho \left(\sigma(\text{stud})_{\text{gen=male}} \right) \right)_{I, A, G}$$

⇒ Model III :- max/min Value Access

① Emp (eid sal), Retrieve eid's who get maximum salary.

⇒ {eid's who gets max salary}

$$\boxed{\begin{aligned} \text{max sal} &\equiv \text{sal} \geq \text{every sal} \\ \text{min sal} &\equiv \text{sal} \leq \text{every sal} \end{aligned}}$$

$$\left\{ \text{eid's whose sal} \right. \\ \left. \geq \text{every sal of emp relation} \right\} \equiv \left\{ \text{eid's of all emp's} \right\} - \left\{ \text{eid's whose sal} \right. \\ \left. < \text{some sal of emp rel} \right\}$$

not max

$$\cdot \pi_{\text{eid}} (\text{Emp}) - \pi_{\text{eid}} \left(\text{Emp} \bowtie_{\text{sal} < S} \rho (\text{Emp}) \right)_{I, S}$$

② Emp(eid, sal) Retrieve eid's who get minimum salary.

⇒ {eid's whose sal
minimum}

⇓

$$\left\{ \begin{array}{l} \text{eid's whose sal} \\ \leq \text{every sal of} \\ \text{Emp relation} \end{array} \right\} \equiv \left\{ \begin{array}{l} \text{all eid's} \\ \text{of Emp} \end{array} \right\} - \left\{ \begin{array}{l} \text{eid's whose sal} \\ > \text{some sal of} \\ \text{Emp Relation} \end{array} \right\}$$

$$\pi_{\text{eid}}(\text{Emp}) - \pi_{\text{eid}} \left(\text{Emp} \bowtie_{\text{sal} > S} \rho(\text{Emp})_{I, S} \right)$$

⇒ Model IV :- Max/Min value Access for each group of records.

① Emp(eid, sal, dno). Retrieve eid's who gets max salary for each dept.

$$\Rightarrow \left\{ \begin{array}{l} \text{eid's who gets} \\ \text{sal} \geq \text{every sal} \\ \text{of his/her dept} \end{array} \right\} \equiv \left\{ \begin{array}{l} \text{eid's of} \\ \text{all Emp's} \end{array} \right\} - \left\{ \begin{array}{l} \text{eid's whose sal} \\ < \text{some emp sal} \\ \text{of higher dept} \end{array} \right\}$$

Emp	eid
e1	
✓e2	
e3	
✓e4	
e5	
✓e6	

-

Emp	Eid	sal	dno
✓	e1	60	2
×	e2	80	2
✓	e3	70	2
×	e4	70	3
✓	e5	60	3
×	e6	50	4

Emp	eid	sal	dno
	e1	60	2
	e2	80	2
	e3	70	2
	e4	70	3
	e5	60	3
	e6	50	4

$$\pi_{\text{eid}}(\text{Emp}) - \pi_{\text{eid}} \left(\text{Emp} \bowtie_{\substack{\text{sal} < S \\ \wedge \text{dno} = D}} \rho(\text{Emp})_{I, S, D} \right)$$

② $\text{stud}(\underline{\text{sid}} \text{ age gen})$ Retrieve sid's whose age min for each gender.

$$\Rightarrow \left\{ \text{sid's whose age} \leq \text{every age of same gender stud's} \right\} \equiv \left\{ \text{sid's of all stud's} \right\} - \left\{ \text{sid's whose age} > \text{some age of same gender stud's} \right\}$$

$$\cdot \pi_{\text{sid}}(\text{stud}) - \pi_{\text{sid}} \left(\text{stud} \bowtie \rho(\text{stud}) \begin{array}{l} \text{age} > A \\ \wedge \text{gen} = G \end{array} \right)$$

③ $\text{Enroll}(\underline{\text{sid}} \underline{\text{cid}} \text{ fee})$, Retrieve sid's who paid minimum fee for each course.

$$\Rightarrow \left\{ \text{sid's who paid fee} \leq \text{every fee of same course} \right\} \equiv \left\{ \text{sid's of Enroll} \right\} - \left\{ \text{sid's who paid fee} > \text{some fee of same course} \right\}$$

Enroll

Sid	Cid	fee
S1	C1	
S2	C1	
S3	C1	
S2	C2	
S3	C2	
S4	C3	

Enroll

Sid	Cid	fee
✓ S1	C1	12
✗ S2	C1	10
✓ S3	C1	14
✓ S2	C2	18
✗ S3	C2	15
✗ S4	C3	20

Enroll

Sid	Cid	fee
S1	C1	12
S2	C1	10
S3	C1	14
S2	C2	18
S3	C2	15
S4	C3	20

$$\cdot \pi_{\text{sid}} \left(\pi_{\text{sidCid}}(\text{Enroll}) - \pi_{\text{sidCid}} \left(\text{Enroll} \bowtie \rho(\text{Enroll}) \begin{array}{l} \text{fee} > F \\ \wedge \text{cid} = C \end{array} \right) \right)$$

Q) stud (sid, sname, age, brach) , Retrieve Snames of students whose branch 'cs' but age not more than 20.

Ans $\pi_{\text{Sname}} \left(\pi_{\text{Sid Sname}} \left(\sigma_{\text{branch}=\text{'cs'}} (\text{stud}) \right) - \pi_{\text{Sid Sname}} \left(\sigma_{\text{age} > 20} (\text{stud}) \right) \right)$

✓ S1 A - S2 A

~~S2 A~~

⇒

<u>Sid</u>	<u>Sname</u>	<u>age</u>	<u>branch</u>
S1	A	19	CS
S2	A	22	CS

* Set Operations :-

⇒ U : Union { : OR }

- : Except/minus { : but not/only }

$\cap$: Intersect { : And }

R.A.

SQL

⇒ R & S must be union compatible to use $R \cup S$, $R - S$, $R \cap S$. } Same in SQL

⇒ R & S union compatible iff

i) Arity of R = Arity of S and

ii) Domain of R Attributes must be with same domain for S attribute respectively.

Eg: ① $\pi_{\text{Sid}(\text{Cid})}(\dots) \cup \pi_{\text{Sid}(\text{Sname})}(\dots)$ X Not allowed

② $\pi_{\text{Sid}}(\dots) \cup \pi_{\text{Sid Sname}}(\dots)$ X Not allowed

③ $\pi_{\text{Sid Cid}}(\dots) \cup \pi_{\text{Cid Sid}}(\dots)$ X Not allowed

④ $\pi_{\text{Sid Sname}}(\dots) \cup \pi_{\text{StudID StudName}}(\dots)$ ✓ Allowed

⑤ $\pi_{\text{Sid Cid}}(\dots) \cup \pi_{\text{Sid Cid}}(\dots)$ ✓ Allowed

⇒ RUS, R-S, RNS set operations
 → Resulted Schema is same as R schema
 → Record set of result distinct.

} Same in SQL set op.

Eg:

R	A	B	C
	4	4	6
	4	4	6
	2	5	5
	2	5	5

S	D	E	F
	2	5	5
	2	5	5
	6	2	3
	6	4	5

* $R \cup S = R$
 recorded (x) result
 if $x \in R$ or $x \in S$

A	B	C
4	4	6
2	5	5
6	2	3
6	4	5

$$\Rightarrow R - (R - S)$$

$$* R \cap S = R$$

record (x) in
result if

$x \in R$ and

$x \in S$

A	B	C
2	5	5

$$* R - S = R$$

record (x) in
result if

$x \in R$ and

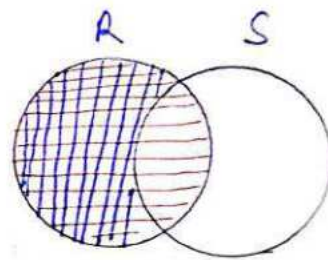
$x \notin S$

A	B	C
4	4	6

$$\Rightarrow R \cap S = R - (R - S)$$

$$R(A \ B) \times S(C \ D)$$

2	4	2	4
3	6	3	7



$R - (R \cap S)$

$$① R \cap S = R - (R - S)$$

$$② R \cap S = \pi_{A,B} \left(\sigma_{R.A=S.C \wedge R.B=S.D} (R \times S) \right)$$

$$③ R \cap S = \pi_{A,B} (R \bowtie S)$$

$R.A = S.C$
 $\wedge R.B = S.D$

$$④ R \cap S = R \bowtie \rho(S)$$

$C \rightarrow A$
 $D \rightarrow B$

* Stud (Sid Sname gen)

S1		F
S2		M
S3		F
S4		M
S5		M

* Course (Cid Cname Instructor)

C1		Karth
C2		Karth
C3		Navathe
C4		Navathe
C5		Ullman

* Enroll (Sid Cid fee)

S1	C1
S1	C2
S1	C3
S1	C5
S2	C2
S3	C2
S3	C5
S4	C5

⇒ model V: Queries uses set operations

1) Retrieve Sid's enrolled some course taught by Karth.

Ans Enroll (Sid Cid) ^{= some} Course (Cid Inst) ^{Karth}

$\pi_{\text{Sid}} \left(\sigma_{\text{Inst} = \text{Karth}} (\text{Enroll} \bowtie \text{Course}) \right)$

OR

$\pi_{\text{Sid}} \left(\text{Enroll} \bowtie \sigma_{\text{Inst} = \text{Karth}} (\text{Course}) \right)$

Sid
S1
S2
S3

2) Retrieve Sid's enrolled for only courses taught by Karth.

Ans { only / But not } = { by using "—" }

$$\left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{some course} \end{array} \right\} - \left\{ \begin{array}{l} \text{Sid's enrolled some} \\ \text{non-Karth course} \end{array} \right\}$$

$$\pi_{\text{Sid}}(\text{Enroll}) - \pi_{\text{Sid}}(\text{Enroll} \bowtie \sigma_{\substack{\text{course} \\ \text{Inst} \neq \text{Karth}}})$$

3) Retrieve Cid's enrolled some female student.

Ans

$$\pi_{\text{Cid}}(\text{Enroll} \bowtie \sigma_{\substack{\text{Stud} \\ \text{gen} = F}})$$

4) Retrieve Cid's enrolled only female students.

Ans

$$\left[\begin{array}{l} \text{Cid's enrolled} \\ \text{some student} \end{array} \right] - \left[\begin{array}{l} \text{Cid's enrolled some} \\ \text{Non-female} \end{array} \right]$$

$$\pi_{\text{Cid}}(\text{Enroll}) - \pi_{\text{Cid}}(\text{Enroll} \bowtie \sigma_{\substack{\text{Stud} \\ \text{gen} \neq F}})$$

5) Retrieve Sid's enrolled some course taught by Karth (OR) some course taught by Navathe.

Ans

$$\pi_{\text{Sid}}(\text{Enroll} \bowtie \sigma_{\substack{\text{course} \\ \text{Inst} = \text{Karth} \vee \\ \text{Inst} = \text{Navathe}}})$$

(OR)

$$\pi_{\text{Sid}} \left(\text{Enroll} \bowtie \sigma_{\text{Inst} = \text{Kanth}}(\text{course}) \right) \cup \pi_{\text{Sid}} \left(\text{Enroll} \bowtie \sigma_{\text{Inst} = \text{Navathe}}(\text{course}) \right)$$

6) Retrieve Cid's enrolled some male and some female student.

Ans

$$\Rightarrow \pi_{\text{Cid}} \left(\text{Enroll} \bowtie \sigma_{\text{gen} = \text{M} \wedge \text{gen} = \text{F}}(\text{Stud}) \right)$$

Empty Result
[Wrong] X
 $\text{gen} = \text{M} \wedge \text{gen} = \text{F}$

$$\Rightarrow \pi_{\text{Cid}} \left(\text{Enroll} \bowtie \sigma_{\text{gen} = \text{M}}(\text{Stud}) \right) \cap \pi_{\text{Cid}} \left(\text{Enroll} \bowtie \sigma_{\text{gen} = \text{F}}(\text{Stud}) \right) \checkmark$$

X

$$\Rightarrow \pi_{\text{Cid}} \left(\sigma \left(\rho(T_1, \text{Enroll}) \times \rho(T_2, \text{Stud}) \times \rho(T_3, \text{Stud}) \right) \right)$$

Empty

$$T_1.\text{Sid} = T_2.\text{Sid} \wedge T_2.\text{gen} = \text{male} \\ \wedge T_1.\text{Sid} = T_3.\text{Sid} \wedge T_3.\text{gen} = \text{female}$$

$$\Rightarrow \pi_{T_1.\text{Cid}} \left(\sigma \left(\rho(T_1, \text{Enroll}) \times \rho(T_2, \text{Stud}) \times \rho(T_3, \text{Enroll}) \times \rho(T_4, \text{Stud}) \right) \right)$$
$$T_1.\text{Sid} = T_2.\text{Sid} \wedge T_2.\text{gen} = \text{male} \wedge \\ T_3.\text{Sid} = T_4.\text{Sid} \wedge T_4.\text{gen} = \text{female} \wedge T_1.\text{Cid} = T_3.\text{Cid}$$

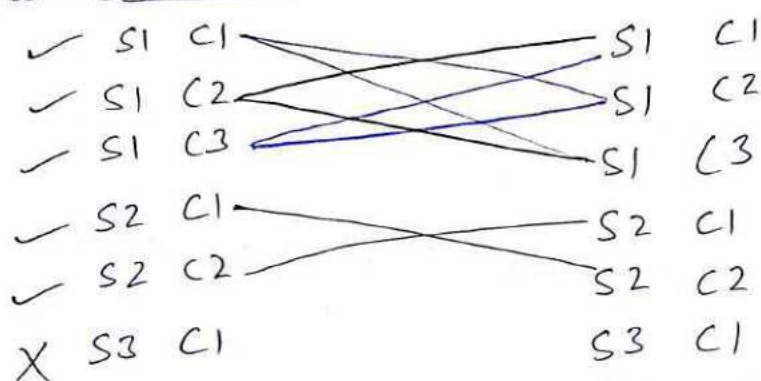
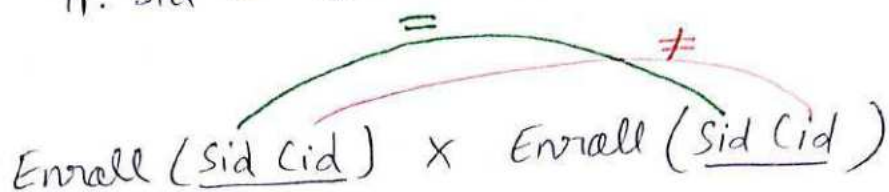
Model:
 $\Rightarrow \forall T \Rightarrow$

Atleast two / Atmost two / only two

i) Enroll(Sid Cid) Retrieve Sid's enrolled Atleast two courses

$\exists T_1 \in \text{Enroll}, \exists T_2 \in \text{Enroll}$ such that

$$T_1.\text{Sid} = T_2.\text{Sid} \wedge T_1.\text{Cid} \neq T_2.\text{Cid}$$



$$\pi_{\text{Sid}} \left(\begin{array}{l} \text{Enroll} \bowtie \rho_{\substack{\text{S,C}}}(\text{Enroll}) \\ \text{Sid} = \text{S} \\ \wedge \text{Cid} \neq \text{C} \end{array} \right)$$

ii) Retrieve Sid's enrolled atleast three courses

$$\pi_{T_1.\text{Sid}} \left(\sigma \left(\rho(T_1, \text{Enroll}) \times \rho(T_2, \text{Enroll}) \times \rho(T_3, \text{Enroll}) \right) \right)$$

$$\begin{array}{l} T_1.\text{Sid} = T_2.\text{Sid} \wedge T_2.\text{Sid} = T_3.\text{Sid} \\ \wedge T_1.\text{Cid} \neq T_2.\text{Cid} \wedge T_2.\text{Cid} \neq T_3.\text{Cid} \\ \wedge T_1.\text{Cid} \neq T_3.\text{Cid} \end{array}$$

iii) Retrieve Sid's enrolled only one course

$$\left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{some course} \\ \text{atleast one} \end{array} \right\} - \left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{atleast two} \\ \text{courses} \end{array} \right\}$$

$$\pi_{\text{Sid}}(\text{Enroll}) - \pi_{\text{Sid}} \left(\text{Enroll} \bowtie \rho(\text{Enroll}) \right. \\ \left. \begin{array}{l} \text{Sid} = S \\ \wedge \text{Cid} \neq C \end{array} \right. \begin{array}{l} S, C \\ S, C \end{array} \right)$$

$\Rightarrow$ Only one $\rightarrow$ [atleast one] but not [atleast two]

iv) Retrieve Sid's enrolled only two courses

$$\left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{atleast two courses} \end{array} \right\} - \left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{atleast three} \\ \text{courses} \end{array} \right\}$$

v) Retrieve Sid's enrolled atmost one course

$$\text{Enroll}(\underline{\text{Sid}} \text{ Cid}) \quad \text{stud}(\underline{\text{Sid}} \text{ Sname})$$

$$\left\{ \begin{array}{l} \text{all Sid's of} \\ \text{Students} \end{array} \right\} - \left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{atleast two courses} \end{array} \right\}$$

$$\pi_{\text{Sid}}(\text{Stud}) - \pi_{\text{Sid}} \left(\text{Enroll} \bowtie \rho(\text{Enroll}) \right. \\ \left. \begin{array}{l} \text{Sid} = S \\ \wedge \text{Cid} \neq C \end{array} \right. \begin{array}{l} S, C \\ S, C \end{array} \right)$$

v) Retrieve Sid's enrolled atmost two courses

$$\left\{ \begin{array}{l} \text{all Sid's} \\ \text{of Students} \end{array} \right\} - \left\{ \begin{array}{l} \text{Sid's enrolled} \\ \text{atleast three} \\ \text{courses} \end{array} \right\}$$

Model:

VII $\Rightarrow$ Max/min number of records in result of $\bowtie$:-

1) Stud (^{PK}Sid Sname) with 100 records

Enroll (^{PK}Sid Cid) with 50 records

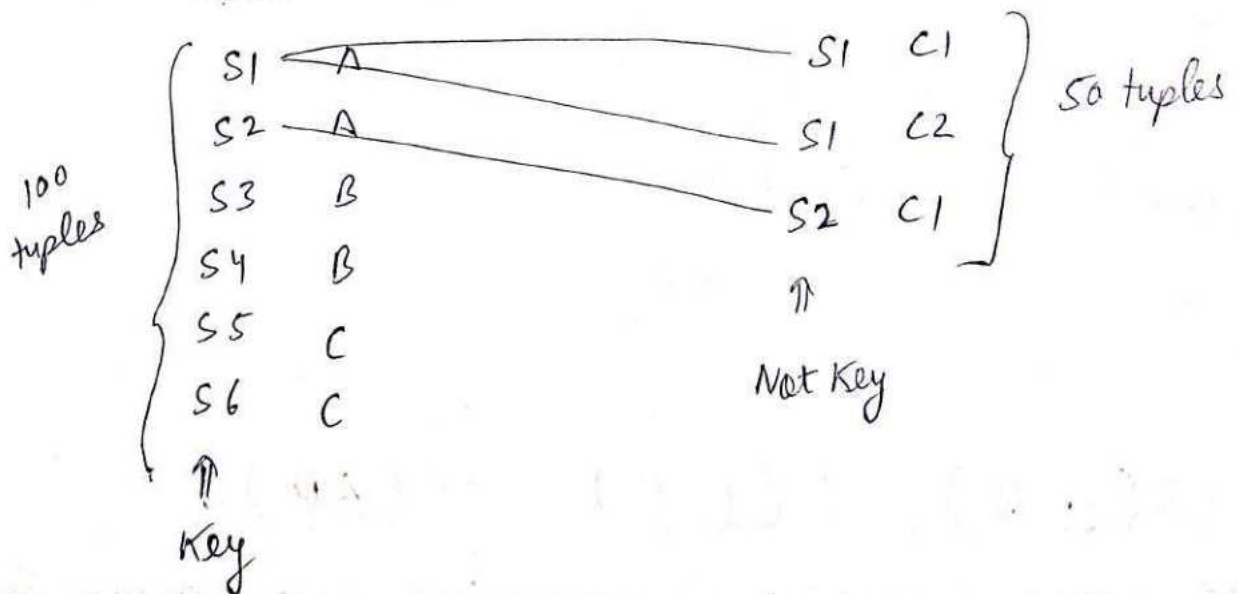
How many (max, min) tuples in result of Stud $\bowtie$ Enroll?

a) (100, 0) ~~b) (50, 50)~~ c) (100, 50)

d) (5000, 0)

Solⁿ

Stud (Sid Sname) $\times$ Enroll (^{FK}Sid Cid)



$\Rightarrow$ max = 50 , min = $\begin{cases} 50 & \text{if FK exists} \\ 0 & \text{if FK not exists} \end{cases}$

II) R : n distinct tuples

S : m distinct tuples

Schema of R

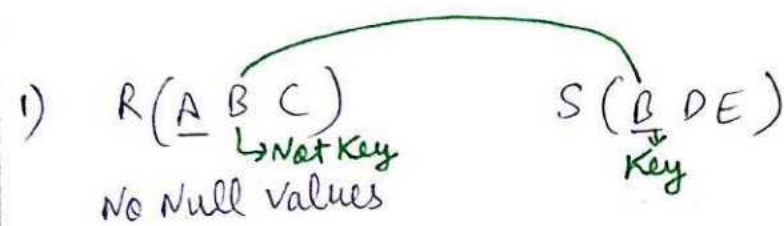
Schema of S

max tuples
 $R \bowtie S$

min tuples
 $R \bowtie S$

with
FK

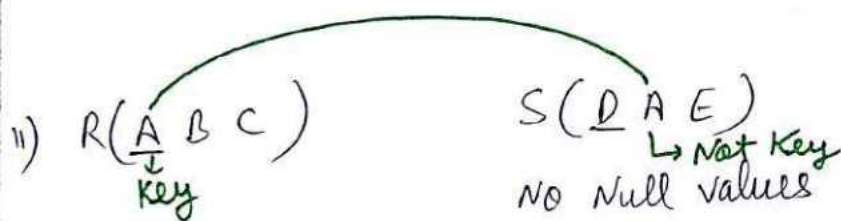
without
FK



n

n

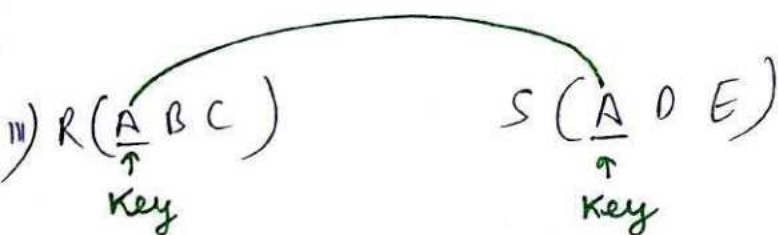
0



m

m

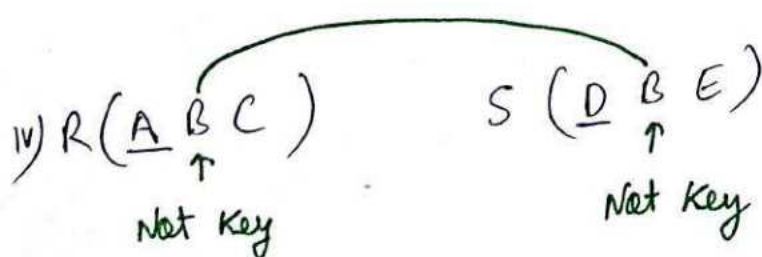
0



$\min(m, n)$

$\min(m, n)$

0



$n * m$

Not possible

0

III) $R(\underline{A} B)$, $S(\underline{D} E)$, $T(\underline{A D})$

with 1000, 2000, 1500 respectively. How many max tuples in result of $R \bowtie S \bowtie T$?

* Expansion of Division:-

$$\left\{ \frac{\pi_{sidCid}(Enroll)}{\pi_{Cid}(Course)} \right\}$$

1) Sid's not enrolled for every course

- [Sid's enrolled proper subset of all courses]

Sid
S2
S3

$$\pi_{Sid} \left(\underbrace{\pi_{Sid}(Enroll) \times \pi_{Cid}(Course)}_{\text{every sid of Enroll rel pairs with every course}} - \underbrace{\pi_{SidCid}(Enroll)}_{\text{Given enrolled records}} \right) : \text{Disqualified records for division}$$

2) Sid's enrolled every course = $\left\{ \text{Sid's enrolled some course} \right\} - \left\{ \text{Sid's not enrolled every course} \right\}$

$$\frac{\pi_{SidCid}(Enroll)}{\pi_{Cid}(Course)} = \frac{\pi_{Sid}(Enroll)}{\pi_{Cid}} - \frac{\pi_{Sid} \left(\pi_{Sid}(Enroll) \times \pi_{Cid}(Course) - \pi_{SidCid}(Enroll) \right)}{\pi_{Cid}}$$

Eg:-

$$1) \frac{\pi_{AB}(R)}{\pi_B(S)} = \frac{\pi_A(R)}{\pi_A} - \frac{\pi_A \left(\pi_A(R) \times \pi_B(S) - \pi_{AB}(R) \right)}{\pi_A} = \begin{bmatrix} A \\ a1 \end{bmatrix}$$

$$2) \frac{\pi_{ABC}(R)}{\pi_C(S)} = \frac{\pi_{AB}(R)}{\pi_{AB}} - \frac{\pi_{AB} \left(\pi_{AB}(R) \times \pi_C(S) - \pi_{ABC}(R) \right)}{\pi_{AB}} = \begin{bmatrix} A & B \\ a1 & b1 \end{bmatrix}$$

$$3) \frac{\pi_{ABCD}(R)}{\pi_{CD}(S)} = \frac{\pi_{AB}(R)}{\pi_{AB}} \left(\frac{\pi_{AB}(R)}{\pi_{AB}} \times \frac{\pi_{CD}(S)}{\pi_{CD}} - \frac{\pi_{ABCD}(R)}{\pi_{ABCD}} \right)$$

$$R/S = \frac{\pi_{R-S}(R)}{\pi_{R-S}} \left(\frac{\pi_{R-S}(R)}{\pi_{R-S}} \times S - \frac{\pi_{R-S,S}(R)}{\pi_{R-S,S}} \right)$$

$$\Rightarrow \frac{\pi_{AB}(R)}{\pi_{BC}(S)} \quad \times \text{ Not Allowed}$$

$$\Rightarrow \frac{\pi_{AB}(R)}{\pi_{AB}(S)} \quad \times \text{ Not Allowed}$$

* Stud (Sid gender)

course (Cid Inst)

Enroll (SidCid)

1) Retrieve Sid's enrolled every course taught by Kerth.

$$\frac{\pi_{SidCid}(Enroll)}{\pi_{Cid} \left(\sigma_{Inst=Kerth}(Course) \right)}$$

$$= \frac{\pi_{Sid}(E)}{\pi_{Sid}} \left(\frac{\pi_{Sid}(E)}{\pi_{Sid}} \times \frac{\pi_{Cid} \left(\sigma_{Inst=Kerth}(C) \right)}{\pi_{Cid}} - \frac{\pi_{SidCid}(E)}{\pi_{SidCid}} \right)$$

Every Sid of enroll rel
pairs with every course

Sid's enrolled
proper subset of Kerth

Enroll

Sid Cid

S1 C1

S1 C2

S1 C3

S1 C4

S2 C2

S2 C4

S3 C4

Course

Cid Inst

C1 Korth

C2 Korth

C3 Korth

C4 Ullman

C5 Navathe

$$Q) \pi_{Cid} \left(\pi_{Sid} \left(\sigma_{gen=fem} (Stud) \right) \times \pi_{Cid} (Enroll) - \pi_{SidCid} (Enroll) \right)$$

what is result of query?

Soln

Every enrolled Cid pair
with all fem student

— Enroll
Records

→ Cid's not enrolled by every female

= Cid's enrolled proper subset of all female students

Ans

* works (eid pid) Project (Pid Pname)

1) Retrieve eid's who works for some DB project.

Project

Pid Pname

P1 DB

P2 DB

P3 DB

P4 OS

P5 NW

$$\pi_{eid} \left(works \bowtie \sigma_{Pname=DB} (Project) \right)$$

2) Retrieve eid's who works for only DB projects.

Ans eid's works for some project - eid's who works for some non DB projects

$$\pi_{eid}(\text{Works}) - \pi_{eid}(\text{Works} \bowtie \sigma_{Pname \neq DB}(\text{Project}))$$

3) Retrieve eid's who works for every DB projects.

Soln

$$\pi_{eidpid}(\text{Works}) / \pi_{pid}(\sigma_{Pname = DB}(\text{Project}))$$

$$= \pi_{eid}(\text{W}) - \pi_{eid}(\pi_{eid}(\text{W}) \times \pi_{pid}(\sigma_{Pname = DB}(\text{Project})) - \pi_{eidpid}(\text{W}))$$

Q) Enroll

Sid	Cid
S1	C1
S1	C2
S2	C2
S3	C1
S4	C2

Course

Cid	Inst
C1	Navathe
C2	Navathe
C3	Ullman

$$\pi_{SidCid}(\text{Enroll}) / \pi_{Cid}(\sigma_{Inst = Korth}(\text{Course}))$$

↑
Empty

How many tuples in result of query?

Solⁿ

$$\pi_{\text{SidCid}}(\text{Enroll}) / \pi_{\text{Cid}}(\sigma_{\text{Inst}=\text{Kanth}}(\text{Course}))$$

↓
Empty

$$= \pi_{\text{Sid}}(E) - \pi_{\text{Sid}} \left(\pi_{\text{Sid}}(E) \times \pi_{\text{Cid}}(\sigma_{\text{Inst}=\text{Kanth}}(\text{Course})) - \pi_{\text{SidCid}}(E) \right)$$

↑ ↑

S1	— empty	S1	×	cid
S2		S2		
S3		S3		
S4		S4		

empty

Ans = 4 //

Q) R with n distinct tuples
X set of Attributes

R --X--

S with m distinct tuples
Y set of Attributes

S --Y--

1) what is Required condition to perform R/S.

Solⁿ • Condition to use R/S : $Y \subset X$

• Resulted Attributes of R/S : $X - Y$

• Cardinality of R/S : $\left\{ 0 \text{ to } \left\lfloor \frac{n}{m} \right\rfloor \right\}$
(min - max tuples)

$$\frac{\pi_{AB}(R)}{\pi_B(S)} = \begin{array}{|c|} \hline A \\ \hline \text{empty} \\ \hline \end{array}$$

$$n \left\{ \begin{array}{l} a_1 b_1 \\ a_1 b_2 \\ a_2 b_3 \\ a_2 b_3 \\ a_3 b_1 \\ a_4 b_2 \end{array} \right. \quad \left. \begin{array}{l} b_1 \\ b_2 \\ b_3 \end{array} \right\} m$$

$$\frac{\pi_{AB}(R)}{\pi_B(S)} = \begin{array}{|c|} \hline A \\ \hline a_1 \\ a_2 \\ \hline \end{array}$$

$$n \left\{ \begin{array}{l} a_1 b_1 \\ a_1 b_2 \\ a_1 b_3 \\ a_2 b_1 \\ a_2 b_2 \\ a_2 b_3 \end{array} \right. \quad \left. \begin{array}{l} b_1 \\ b_2 \\ b_3 \end{array} \right\} m$$

$$\uparrow$$

$$\left[\frac{n}{m} \right]$$

NOTE:- If relation R with n distinct tuples & relation S with 0 tuples then number of records of R/S result are atmost n records.

*) Relation R with n distinct records
Relation S with m distinct records

RA Expression	Cardinality (min-max) ^{Tuples}	Commutative?
1) $\pi_A(R)$	Atmost n [# of distinct val of Attr A]	No
2) $\sigma_c(R)$	{ 0 to n }	Yes
3) $R \times S$	$n \times m$	Yes
4) $R \bowtie S, R \bowtie_c S$	{ 0 to $n \times m$ }	Yes
5) $R \bowtie S$	{ n to $n \times m$ }	No
6) $R \bowtie S$	{ m to $n \times m$ }	No

7) $R \bowtie S$	$\{\max(n,m) \text{ to } n \times m\}$	Yes
8) $R \cup S$	$\{\max(n,m) \text{ to } n+m\}$	Yes
9) $R \cap S$	$\{0 \text{ to } \min(n,m)\}$	Yes
10) $R - S$	$\{0 \text{ to } n\}$	No
11) R/S	$\{0 \text{ to } \lfloor \frac{n}{m} \rfloor\}$	No

* SQL - Structured Query Language :-

⇒ Sub Languages of SQL :- DDL, DML, DCL, TCL

Data Definition Language (DDL)	Data Manipulation Language (DML)	Data Control Language (DCL)
Used to define/modify definition or structure of DB table also used to modify Integrity Constraints ⇒ CREATE TABLE ⇒ DROP TABLE ⇒ ALTER TABLE [To add/remove attributes] ⇒ To define Primary Key, UNIQUE, NOT NULL, FOREIGN KEY etc.	Used to modify data records of the table, also used to access data from DB Tables ⇒ INSERT INTO ⇒ DELETE FROM ⇒ UPDATE SET ... ⇒ SELECT * FROM table WHERE condition } data access	Used to grant data Access & revoke data Access to Users (Security) ⇒ GRANT ⇒ REVOKE Transaction Control Language (TCL) Used for concurrency control to avoid inconsistency ⇒ COMMIT ⇒ ROLL BACK ⇒ CHECK POINT

SQL Query

③ SELECT DISTINCT $A_1, A_2, \dots, A_n \Rightarrow$ Projection (π)

① FROM $R_1, R_2, \dots, R_m \Rightarrow$ Cross Product ($\times$)

② WHERE $P; \Rightarrow$ selection operator (σ)

equal

RA Query

$$\pi_{A_1, \dots, A_n} \left(\sigma_P (R_1 \times R_2 \times \dots \times R_m) \right)$$

*)

Catalog (<u>Sid</u> <u>Pid</u>)		Parts (<u>Pid</u> <u>Col</u>)	
S1	P1	P1	Red
S1	P2	P2	Red
S2	P2	P3	Green
S3	P3		

1) Retrieve sid's who supplied some red part.

RA $\Rightarrow$ $\pi_{\text{Sid}} \left(\text{Catalog} \bowtie \sigma_{\text{Col}=\text{Red}} (\text{Parts}) \right)$

SQL $\Rightarrow$ ③ SELECT Sid / {SELECT DISTINCT Sid} to eliminate repeated values from result

① FROM Catalog AS C, Parts P

② WHERE C.pid = P.pid and color = "Red";

=

Sid
S1
S1
S2

Result

NOTE: ① P : "AS" $\Rightarrow$ Used for Rename of table or attributes

② π : SELECT DISTINCT

*) $R \Rightarrow n \text{ tuples \& } X \text{ \# of Attr.}$
 $S \Rightarrow m \text{ tuples \& } Y \text{ \# of Attr.}$

Select *

FROM R, S;

$\Rightarrow$ Arity of SQL Query :- $X + Y$

Cardinality of SQL Query :- $n * m$

*) $\bowtie_c$: Conditional Join

$\bowtie$: Natural Join

$\Join_L$: Left Outer Join

$\Join_R$: Right Outer Join

$\Join_{full}$: Full Join

Eg:- $R(\underline{AB}) \quad S(\underline{BC})$

$R \bowtie S \Rightarrow \text{SELECT } *$

FROM R NATURAL JOIN S;

$\Rightarrow \text{SELECT } *$

FROM R JOIN S ON $R.B = S.B$;

$\Rightarrow$ SELECT *
 FROM R, S
 WHERE R.B = S.B ;

* Basic SQL clauses :-

$\Rightarrow$ ⑤ SELECT [DISTINCT] ^⑥ A₁, A₂, ----- A_n

① FROM R₁, R₂, ----- R_m

② [WHERE condition]

③ [GROUP BY Attributes]

④ [HAVING condition]

⑦ [ORDER BY Attributes [DESC]] ;

[Execution flow of SQL Query :-]

↓
 FROM
 ↓
 WHERE
 ↓
 GROUP BY
 ↓
 HAVING
 ↓
 SELECT
 ↓
 DISTINCT
 ↓
 ORDER BY

*) SELECT } mandatory clauses to
 FROM } write SQL Query

Eg:- DUAL : Dummy table

$\rightarrow$ SELECT sysdate
 FROM DUAL ;

$\rightarrow$ SELECT 2+5
 FROM DUAL ; = 7

* Aggregate Function of SQL :-

⇒ SQL supports five aggregate functions :-

Count(), Sum(), Avg(), Min(), Max().

⇒ Aggregate functions computes aggregation of Non-NULL values.

Eg:-

R	A	B
	20	10
	60	NULL
	NULL	15
	20	20
	80	NULL
	NULL	25
	50	30

) Select Count()
From R;

of records

Count(*)
7

) Select Count() A1, Count(A) A2,

of Non-Null values of A

Count(DISTINCT A) A3

FROM R;

of distinct Non-Null values of A

A1	A2	A3
7	5	4

*) Select sum(A) A1, sum(DISTINCT A) A2, AVG(A) A3,

AVG(DISTINCT A) A4, Min(A) A5, Max(A) A6

FROM R;

A1	A2	A3	A4	A5	A6
230	210	46	52.5	20	80

Ans

*) Select *

FROM R

where $[A = \max(A)]$;

for each record

Ans

→ $20 = \max(20) \checkmark$

$60 = \max(60) \checkmark$

$NULL = \max() \times$

$20 = \max(20) \checkmark$

$80 = \max(80) \checkmark$

$NULL = \max() \times$

$50 = \max(50) \checkmark$

{ There is no benefit
of using aggregation
in where clause }

{ Use aggregation in
select & Having clause }

*) Select Avg(A), B

FROM R;

Invalid Expression, Result is not in proper form.

Soln

Avg(A)	B
46	☐
	☐
	☐
	...
	☐

↓
Scalar
value

↑
set of values

×

* GROUP BY Clause :-

⇒ Used to group data records of relation based on values of group by attributes.

Eg:-

R	A	B	C
	a1	b1	90
	NULL	b5	40
	a3	b2	95
	NULL	b5	50
	a1	b1	80
	a2	b3	90
	a3	b2	90
	a1	b2	NULL
	a2	b3	50

Group by (A)
(4 group's)

Group by (A,B)
(5 group's)

R	A	B	C
	a1	b1	90
	a1	b1	80
	a1	b2	NULL
	NULL	b5	40
	NULL	b5	50
	a2	b3	90
	a2	b3	50
	a3	b2	95
	a3	b2	90

⇒ SELECT X, AVG(Y)
FROM R
GROUP BY X;

↑
for each group

of records in result = # of groups

Eg:- SELECT A
FROM R
GROUP BY A;

A
a1
NULL
a2
a3

NOTE:- If group by clause used in query then :-

- i) Only attributes of group by is allowed in select class.
- ii) Allowed aggregate func. in select class & computes aggregation for each group.
- iii) Not allowed to select any other attribute in select class other

than attributes of group by.

Eg:- 1) Select A
FROM R
Group by A;

=

A
a1
NULL
a2
a3

ii) Select Avg(c)
FROM R
Group by A;

=

Avg(c)
85
45
70
92.5

iii) Select A, Avg(c)
FROM R
Group by A;

=

A	Avg(c)
a1	85
NULL	45
a2	70
a3	92.5

iv) Select A, B
FROM R
Group by (A, B);

=

A	B
a1	b1
a1	b2
NULL	b5
a2	b3
a3	b2

v) Select B
FROM R
Group by A;

Invalid

vi) Select A, B
FROM R
Group by A;

Invalid

* Having Clause :-

⇒ Used to select the group based on having clause condition.

⇒ WHERE clause condition tested for each record of FROM clause, Having clause condition tested for Each GROUP of GROUP BY clause.

⇒ Having clause condition must be over aggregate function or functions like some(), every(), etc. but not allowed direct attribute comparison.

Eg:-

i) ④ Select A

① FROM R

② Group by A

③ having Avg(c) > 80;

=

A
a1
a3

ii) Select A

FROM R

Group by A

having some(c) > 75;

=

A
a1
a2
a3

iii) Select A

FROM R

Group by A

having every(c) > 75;

=

A
a3

IV) Select A
FROM R
Having Avg(C) > 75;

Allowed

each record
to be tested

⇓

Select A
FROM R
where Avg(C) > 75;

NOTE:-

If having clause used without group by having clause condition tested for each group & each record considered as one group.

V) Select A
FROM R
Group by A
having C > 50;

Invalid

* Comparison with NULL :-

Odd Rule

⇒ NULL is not zero, not empty.

X
NULL

unknown value

⇒ NULL is set of ASCII values assigned by DBMS such that values of two NULL's may not same.

⇒ Comparison with NULL is treated as unknown [Neither False nor True].

Eg:-

$NULL = 5$
 $NULL \neq 5$
 $NULL \leq 5$
 $NULL > 5$
 $NULL = NULL$

} unknown

⇒ R

A	B
a1	60
a2	NULL
a3	20
a4	NULL

Select *
 from R
 where B > 50;

T (for a1, 60)
 U (for a2, NULL)
 F (for a3, 20)
 U (for a4, NULL)

where clause discard the record if condition is false or unknown.

⇒ Emp (eid ename ppno)

Retrieve eid's who have no passport

Soln

Emp	(<u>eid</u>)	ename	ppno
e1	A	P5	X
e2	B	NULL	✓
e3	C	P1	X
e4	D	NULL	✓
e5	E	P3	X

* Select eid
 FROM Emp
 where ppno is NULL;

↑
True if ppno val NULL

→ "Is" clause to access data record based on NULL values

- IS NULL
- IS NOT NULL

(OR) Select eid
 FROM Emp
 Group by eid
 Having count(pno) = 0;

⇒ SQL Comparisons are 3 value truth table
 [False, True, Unknown]

$(A > 20)$ and $(B = 5)$
 T/F/U T/F/U

OR	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

AND	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

False = 0

Unknown = $\frac{1}{2}$

True = 1

$X \text{ OR } Y = \max(X, Y)$

$X \text{ AND } Y = \min(X, Y)$

NOT	
T	F
F	T
U	U

⇒

R	A	B	C	
U	20	NULL	10	X
U	NULL	20	10	X
F	4	15	NULL	X
T	15	25	15	✓
U	NULL	5	10	X

• select *
from R

where $(A > 5)$ and $(A > B \text{ OR } B > 5)$

• How many records in result?

Soln $(A > 5)$ and $(A > B \text{ OR } B > 5)$

T and (U OR U)

U and (U OR T)

F and ()

✓T and (F OR T)

U and (U OR F)

Ans = 1.

⇒ Arithmetic op% with NULL values remains NULL.

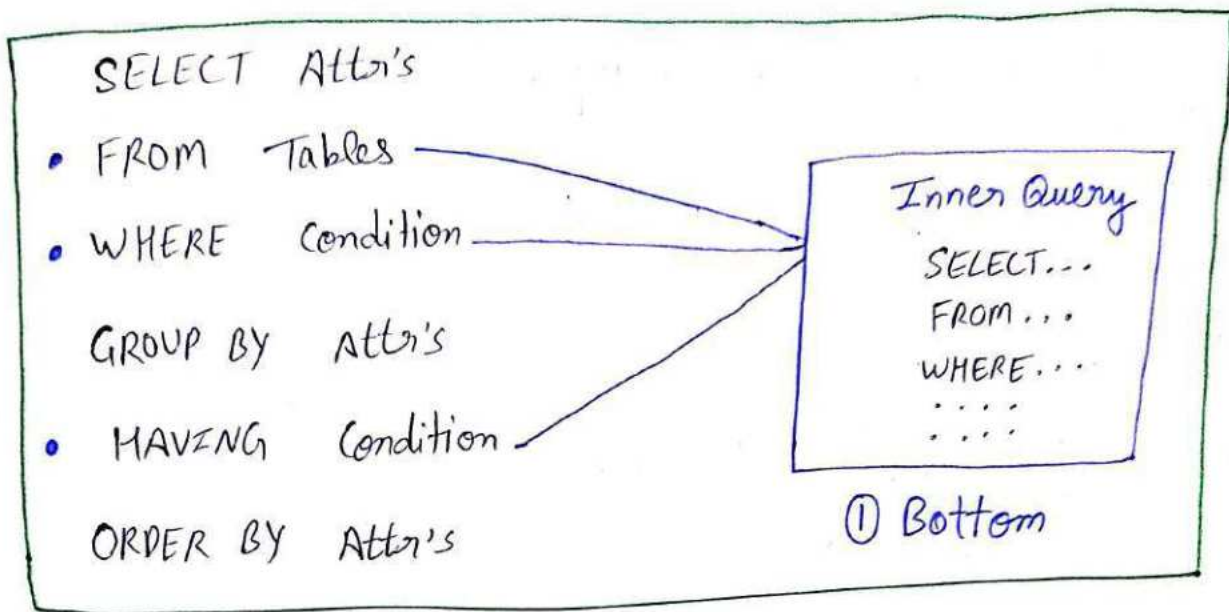
$$\text{NULL} + 5 = \text{NULL}$$

$$\text{NULL} * 5 = \text{NULL}$$

* Nested Queries :-

① Non - Correlated Nested Queries :-

⇒ Inner Query Independent of Outer Query



② Top

Eg:-

Emp (eid, sal, gen)

Select eid

From Emp

where sal > (

① Bottom

select Avg(sal)

From Emp

where gen = female)

;

② Top

② Co-related Nested Queries :-

⇒ Inner Query not independent of Outer Query.

⇒ Co-related nested Query is allowed in where clause, Having clause of outer Query.

Eg:-

Select *

Top ① From R

Top ③ where (Select count(*)

Bottom ② From S
where $R.A > S.B$) < 3 ;

R	A	S	B	R.A = 60	70	20	90	80	
✓	60		90	X	X	X	X	X	} R.A > S.B
✓	70		70	X	X	X	✓	✓	
✓	20		20	✓	✓	X	✓	✓	
X	90		60	X	✓	X	✓	✓	
X	80		80	X	X	X	✓	X	
				Count = 1	2	0	4	3	} < 3

=

A
60
70
20

O/P.

⇒ Top-bottom-Top for each record.

NOTE:-

⇒ If co-relation in where clause, inner query recomputes for each record of outer query.

⇒ If co-relation in having clause, inner query recomputes for each group of outer query.

Q) Emp (eid sal)

1) Retrieve eid's who gets highest salary.

*** a) $\pi_{eid}(EMP) - \pi_{eid} \left(Emp \bowtie_{sal < S} \rho_{T,S}(Emp) \right) \quad - : \text{EXCEPT/MINUS}$

⇒ SQL Query without using aggregate

```
SELECT eid
FROM Emp
EXCEPT
SELECT T1.eid
FROM Emp T1, Emp T2
where T1.sal < T2.sal ;
```

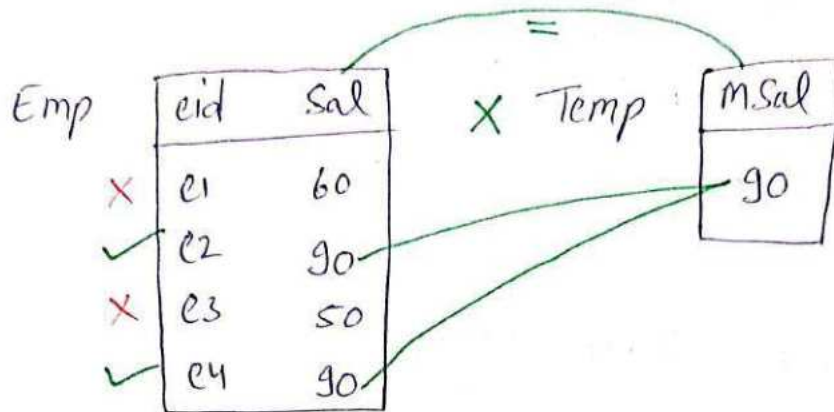
b) using max() :-

```
Select eid
From Emp
where sal = ( Select max(sal)
              FROM Emp );
```

⇒ SELECT eid

FROM Emp, (SELECT max(sal) mSal
FROM Emp) Temp

where Sal = mSal ;



2) Retrieve eid's who gets 2nd highest salary.

a) without using aggregate function :-

$\rho(\text{Temp}, \pi_{\text{eid sal}} \left(\text{Emp} \bowtie_{\text{sal} < S} \rho(\text{Emp})_{I, S} \right)$
 ↑
 emp's whose sal not max

$\pi_{\text{eid}}(\text{Temp}) - \pi_{\text{eid}} \left(\text{Temp} \bowtie_{\text{sal} < S} \rho(\text{Temp})_{I, S} \right)$

⇒ WITH Temp (eid, sal) AS

(SELECT distinct T1.eid, T1.sal
FROM Emp T1, Emp T2
where T1.sal < T2.sal)

SELECT eid
FROM Temp
EXCEPT

Select T1.eid

FROM Temp T1, Temp T2

where T1.Sal < T2.Sal ;

* WITH clause :-

⇒ Subquery result of WITH clause can be reused many times for query processing so that it avoids recomputations.

b) Using max () :-

SELECT eid

FROM Emp

WHERE Sal = (Select max(sal)
FROM Emp

where sal < (Select max(sal)
FROM EMP)) ;

* Co-related nested query :-

Emp
T1

eid	Sal
e1	80
e2	80
e3	70
e4	70
e5	50
e6	30

Emp
T2

eid	Sal
e1	80
e2	80
e3	70
e4	70
e5	50
e6	30

Count # of
distinct T2.Sal
which are greater
than T1.Sal

SELECT eid $\longrightarrow$ K^{th} highest sal

Top ① FROM Emp T1

Top ③ WHERE ($\begin{array}{l} \text{Select count (DISTINCT T2.Sal)} \\ \text{FROM Emp T2} \\ \text{where T2.Sal > T1.Sal} \end{array} \text{) = } \underbrace{(K-1)}_1 ;$

For 2nd highest sal

- I) Retrieve eid's who gets 5th highest salary.
- II) Retrieve eid's who gets 5 highest salary.
- III) Retrieve eid's who gets 5th least salary.
- IV) Retrieve eid's who gets 5 least salaries.

* Select *
 From Emp
 where sal > (select sal
 FROM Emp
 where dno = 5) ;

ERROR

↓

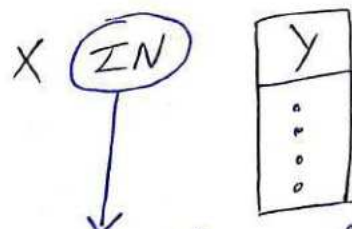
* Functions used for Nested Queries :-

Functions :-

- | | | |
|------------------------|---|--|
| 1) IN / NOT IN | } | suitable for
Non-Correlated
Nested Queries |
| 2) ANY | | |
| 3) ALL | | |
| 4) EXISTS / NOT EXISTS | } | suitable for Co-related
Nested Q. |

⇒ IN Function :-

⇒ used for membership test, it means record belongs to the set or not.



TRUE if x value is member of Y set.

[x value is equal to some value of Y set]

⇒ [IN func. can be used for equijoin ($\bowtie$) Queries]

Eg 0 ① $R(\underline{A}B) \quad S(\underline{B}C)$ = Some

$\pi_{A,B} (R \bowtie S) \equiv \text{Select } * \text{ FROM } R \text{ where } R.B \text{ IN } \left(\begin{array}{|l|} \hline \text{Select } B \\ \text{FROM } S \\ \hline \end{array} \right);$ = Some

② $R(AB) \quad S(CD)$

$\pi_{AB} \left(\begin{array}{l} R \bowtie S \\ R.A = S.C \\ \wedge R.B = S.D \end{array} \right) \equiv \text{Select } * \text{ FROM } R \text{ where } (R.A, R.B) \text{ IN } \left(\begin{array}{|l|} \hline \text{select } C, D \\ \text{FROM } S \\ \hline \end{array} \right);$ = Some

III) Emp (eid FK ename supID)

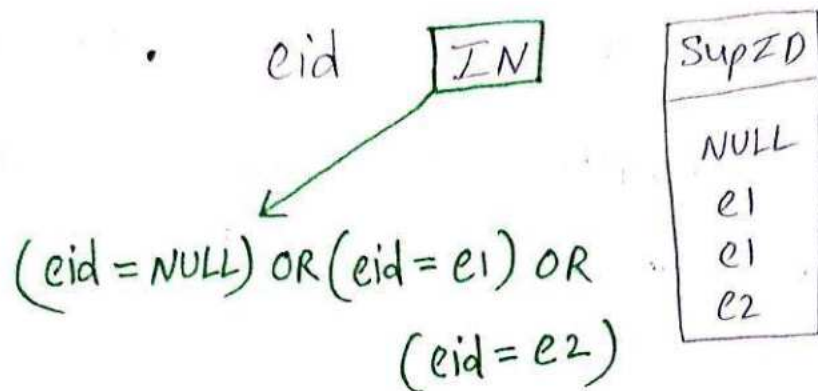
a) Retrieve names of supervisors. = Some

Soln

Emp (<u>eid</u> ename supID)			X Emp (<u>eid</u> ename supID)		
e1	A	NULL	e1	A	NULL
e2	B	e1	e2	B	e1
e3	C	e1	e3	C	e1
e4	D	e2	e4	D	e2

$\xRightarrow{R.A.} \pi_{ename} \left(\begin{array}{l} \text{Emp} \bowtie \rho(\text{Emp}) \\ \text{eid} = S \end{array} \right)$

⇒ Select ename
FROM Emp
where eid IN (Select supID
FROM Emp);



b) Retrieve names of emp's who are not supervisors.

Soln

R.A. ⇒ ≠ Every

{ All emp's } - { emp's who are equal to some supervisors }

IN ≡ { = some }

NOT IN ≡ { ≠ Every }

$\pi_{ename} \left(\pi_{Emp} - \pi_{eid,ename} \left(Emp \bowtie_{eid=S} \rho_{Emp} \left(\begin{matrix} I, N, S \end{matrix} \right) \right) \right)$

SQL ⇒ Select ename
FROM Emp

where eid NOT IN (Select SupID
FROM Emp
where supID IS NOT NULL);

~~(eid ≠ NULL)~~ and (eid ≠ e1) and (eid ≠ e2)

SupID
NULL
e1
e1
e2

NOTE :-

- $\Rightarrow$ IN func. (= Some) is behaves like OR connectivity with equal comparison for inner query result.
- $\Rightarrow$ NOT IN func. ($\neq$ Every) is behaves like AND connectivity with Not equal comparison of inner query records.
- $\Rightarrow$ If inner query result consists NULL value, NOT IN func. never be True. It can be false or unknown.

$\Rightarrow$ ANY and ALL Function :-

* Preceded by comparison :- $<$, $\leq$, $>$, $\geq$, $<>$ ↗ Not equal

$X > \text{ANY}$

Y
⋮

↙
TRUE if $X > \text{Some } Y$
 $\bowtie_c$

$X > \text{ALL}$

Y
⋮

↙
TRUE if $X > \text{every } Y$

[ANY func. can used for conditional Join ($\bowtie_c$) Queries]

Eg:- $R(\underline{A}) S(\underline{B})$ Retrieve 'A' value of R more than some B value of S

Model 1 :- $\pi_A (R \bowtie_{R.A > S.B} S)$: Select A
FROM R
where $R.A > \text{ANY} (\text{SELECT } B \text{ FROM } S);$

⇒ Retrieve 'A' value of R more than every 'B' of Rel S.

model 2 :- $\pi_A(R) - \pi_A \left(R \bowtie_{R.A \leq S.B} S \right)$
of RA

* Select A
From R

where $R.A > \text{ALL} \left(\begin{array}{l} \text{Select B} \\ \text{FROM S} \end{array} \right);$

NOTE :-

- * ~~Any~~ ^{'ANY'} funx. behaves as 'OR' connectivity with given comparison of inner query result.
- * 'ALL' funx. behaves as 'AND' connectivity with given comparison of inner query result.
- * If inner query result consists 'NULL' value, 'ALL' funx. over inner query result never be TRUE.

$$\left[\begin{array}{l} \text{IN} \iff \{ = \text{ANY} \} \\ \text{NOT IN} \iff \{ \neq \text{ALL} \} \end{array} \right]$$

⇒ EXISTS / NOT EXISTS Function :-

- * EXISTS funx. used to test result of inner query is empty or not empty.
- * If result of inner query not empty EXISTS funx. returns TRUE.

EXISTS (Inner Q)

↑

TRUE if Inner Q result not empty

- Some record in result of Inner Q
- used for (MC) Queries

Eg:-

$$\Rightarrow \pi_{R.A} \left(R \bowtie_{R.A > S.B} S \right)$$

Select *

① FROM R

③ where exists (Select *
FROM S
where R.A > S.B) ;

not empty

②

Q) Emp (eid, sal, dno)
1) Retrieve eid's whose salary less than some Emp sal of dept 5.

Soln

$$RA :- \pi_{eid} \left(Emp \bowtie_{sal < S} \rho \left(\sigma_{\substack{dno=5 \\ Z, S, D}} (Emp) \right) \right)$$

SQL: ① SELECT DISTINCT T1.eid
FROM Emp T1, Emp T2
WHERE T1.sal < T2.sal
and T2.dno = 5;

② Using ANY funx. :-

```
SELECT eid
FROM Emp
WHERE sal < ANY ( select sal
                  FROM Emp
                  where dno = 5 );
```

③ Using EXISTS funx. :-

```
SELECT eid
FROM Emp T1
WHERE EXISTS ( SELECT *
               FROM Emp T2
               where T2.dno = 5
               and T2.sal > T1.sal );
```

11) Retrieve eid's whose sal less than every emp sal of dept 5.

Relm

RA. $\Rightarrow \pi_{eid}(Emp) - \pi_{eid} \left(Emp \bowtie \rho \left(\sigma_{\substack{dno=5 \\ T_1, S, D}}(Emp) \right) \right)$

SQL :- ① select eid

From Emp

EXCEPT

Select Distinct T1.eid

FROM Emp T1, Emp T2

where T1.sal $\geq$ T2.sal

and T2.dno = 5;

② Using ALL func. :-

select eid

FROM Emp

where sal < ALL (select sal
FROM Emp
where dno = 5);

③ Using EXISTS func. :-

select eid

FROM Emp T1

where NOT EXISTS (empty
select *
FROM Emp T2
where T2.dno = 5
and T2.sal <= T1.sal);

not (T1.sal ≥ some sal of dno = 5)

= T1.sal < every sal of dno = 5

EXISTS : Some / any / atleast one ($\exists$)
NOT EXISTS : Every / all ($\forall$)

Q) Stud (sid gen age)

1) Retrieve sid's of female students whose age more than age of every male student.

Soln

$$\underline{RA} \Rightarrow \pi_{sid} \left(\sigma_{gen=fem} (stud) \right) - \pi_{sid} \left(stud \bowtie \rho_{\substack{gen=fem \\ G=male \\ age \leq A}} (stud) \right)$$

SQL :- ① using ALL func. :-

```
Select Sid
From stud
where gen = fem
and age > ALL ( select age
                From stud
                where gen = male );
```

② Using NOT EXISTS func. :-

```
Select Sid
FROM stud T1
where T1.gen = fem
and NOT EXISTS ( select *
                  FROM stud T2
                  where T2.gen = male
                  and T1.age ≤ T2.age );
```

not ($T1.age \leq \text{age of some male}$)

$\equiv T1.age > \text{age of every male}$

Q)

R	A	B
	a1	10
	a2	12
	a3	5
	a4	8

S	C	D
	5	2
	10	4
	15	6
	2	8

How many tuples in result?

i) Select * { if False $\Rightarrow$ Empty always }
 FROM R
 where $B > \text{ANY}$ ($\downarrow$ Select C
FROM S
where $D > 10$);

Soln

$B > \text{ANY}$

C

 } empty

$\Rightarrow \pi_{AB} \left(R \bowtie_{R.B > S.C} \sigma_{\substack{(S) \\ D > 10}} \right)$
 (0 Tuples) empty

Ans = 0 tuples

ii) Select *
 FROM R { if TRUE $\Rightarrow$ Empty always }
 where $B > \text{ALL}$ ($\downarrow$ Select C
FROM S
where $D > 10$);

Soln

$B > \text{ALL}$

C

 } Empty

Ans = 4 Tuples

$\Rightarrow \pi_{AB}(R) - \pi_{AB} \left(R \bowtie_{R.B \leq S.C} \sigma_{\substack{(S) \\ D > 10}} \right)$
↑ empty empty

iii) Select *
FROM R
Where EXISTS (

TRUE
↓

Not Empty

Select count (*)
FROM S
where S.D > 10
and R.B > S.C);

Count(x)
0

Not empty

Ans = 4 tuples //

[$\Rightarrow$ Count func. never results empty set.]

Q) R

A	B
a1	NULL
a2	NULL

How many of given Q are results empty set?

a) Select B
FROM R;

✓ c) Select Sum(B) Empty //
FROM R;

b) Select Count(B)
FROM R;

✓ d) Select min(B) Empty //
FROM R;

Soln a)

B
NULL
NULL

} not empty

c)

Sum(B)

} Empty

b)

Count(B)
0

} not empty

d)

min(B)

} Empty

NOTE:-

$\Rightarrow$ Count func. results always non-empty record set.

Q) Emp (eid sal dno)

1) Retrieve eid's who gets highest salary for each dept.

Soln

$$\pi_{\text{eid}} (\text{eid}) - \pi_{\text{eid}} \left(\begin{array}{l} \text{Emp} \bowtie \rho(\text{Emp}) \\ \text{sal} < \text{sal} \\ \wedge \text{dno} = \text{dno} \end{array} \right)$$

$\Rightarrow \text{sal} \geq \text{every sal of same dept.}$

* select eid

FROM Emp T₁

where sal $\geq$ ALL (select sal
FROM Emp T₂
where T₂.dno = T₁.dno);

(OR)

$\Rightarrow$

Emp (eid sal dno) X Temp (dno msal)

e1	40	2	2	80
e2	80	2		
e3	60	2		
e4	70	3	3	70
e5	60	3		

Preferred

* select eid

FROM Emp, (select dno, max(sal) msal
FROM Emp
Group by dno) temp

Where Emp.dno = temp.dno

and sal = msal ;

Q) family (parent, child, childDOB)

1) Retrieve youngest child of each parent.

Soln

family (parent, child, childDOB)	X Temp (parent, max_childDOB)
A B 1990	A 1992
A C 1997	B 2022
B D 2020	
B E 2022	

* Select child
FROM family
where (parent, childDOB) <= (Select parent, max(childDOB)
FROM family
Group by parent);

Q) Works (eid, pid)

Retrieve eid's who works for only one projects.

1) * eid's works for some project — eid's works for atleast two projects

→ Select eid
FROM works
EXCEPT
Select T₁.eid
FROM works T₁, works T₂
where T₁.eid = T₂.eid and
T₁.pid <> T₂.pid;

2)* works (eid pid)

X	e1	p1
	e1	p2
	e1	p3
X	e2	p1
	e2	p2
✓	e3	p1
✓	e4	p3

select eid

FROM works

Group by eid

having count(*) = 1;



Every Query having clause condition over aggregation can re-write by using where clause.

(OR)

⇒ select eid

FROM (select eid, count(*) C
FROM works
Group by eid) Temp

where C = 1;

eid	count(*)
e1	3
e2	2
e3	1
e4	1

Q) Emp (eid gen dno sal)

Retrieve dept no for which avg female emp's sal of the dept is more than avg sal of male emp's of company.

Sal

Emp	(<u>eid</u> gen dno sal)
	e1 Fem 2 80
	e2 Fem 2 90
	e3 male 2 70
	e4 male 3 80
	e5 male 3 90
	e6 Fem 3 70

⇒

eid	gen	dno	sal
e1	Fem	2	80
e2	Fem	2	90
e6	Fem	3	70

⇒ ⑤ Select dno

① FROM Emp

② where gen = female

③ Group by dno

④ having Avg(sal) > (Select Avg(sal)
FROM Emp
where gen = male) ;

Avg sal of all
male emp's

Avg sal of female emp's
for each dept.

(OR) ⇒ Select dno

FROM (Select dno, Avg(sal) S
FROM Emp
where gen = female
Group by dno) Temp

where S > (select Avg(sal)
FROM Emp
where gen = male) ;

11) Retrieve dno for which avg sal of the female emp's of the dept is more than avg sal of male emp's of the same dept.

Soln

Select dno

FROM Emp T1

where T1.gen = female

Top

① Group by $T_1.dno$

③ having $Avg(sal) >$

for each dept Avg sal
of female emp's

② Bottom

```

select Avg(sal)
FROM Emp T2
where T2.gen = male
and T2.dno = T1.dno
);

```

(OR)

$\Rightarrow$ Select dno

Top ① FROM (select dno, Avg(sal) S
FROM Emp
where gen = female
Group by dno) Temp

Top ③ where S >

Fem avg sal
of each dept

```

select Avg(sal)
FROM Emp
where Emp.gen = male
and Temp.dno = Emp.dno
);

```

② Bottom

(OR)

$\Rightarrow$ Select Temp1.dno

```

FROM ( select dno, Avg(sal) FAvg
FROM Emp
where gen = fem
Group by dno ) Temp1,

```

```

( select dno, Avg(sal) MAvg
FROM Emp

```


where gen = male

Group by dno) Temp 2

where Temp1.dno = Temp2.dno

and Favg > MAvg ;

* SQL Query equal to Division of RA :-

$\Rightarrow$ Enroll (Sid Cid) course (Cid Inst)

Retrieve sid's enrolled for every course taught by Korith.

$\Rightarrow \pi_{\text{Sid Cid}}(\text{Enroll}) / \pi_{\text{Cid}}(\sigma_{\text{Inst}=\text{Korith}}(\text{course}))$

$\Rightarrow$ SQL Co-related Query :-

Enroll
 T_1

Sid	Cid
S1	C1
S1	C2
S1	C3
S1	C4
S2	C2
S2	C4
S3	C4

course

Cid	Inst
C1	Korith
C2	Korith
C3	Korith
C4	Navathe
C5	Navathe

Enroll
 T_2

$\emptyset$
empty

Sid	Cid
S1	C1
S1	C2
S1	C3
S1	C4
S2	C2
S2	C4
S3	C4

Retrieve T_1 .sid of Enroll(T_1)

If { All cid's of Korith } - { cid's of Enroll(T_2) which are enrolled by T_1 .sid } = $\emptyset$ empty

* Query ①: Select distinct $T_1.sid$

Top ① FROM Enroll T_1

Top ③ WHERE NOT EXISTS (

(Select distinct sid Enroll) T_1
To avoid recomputation of each record

Empty

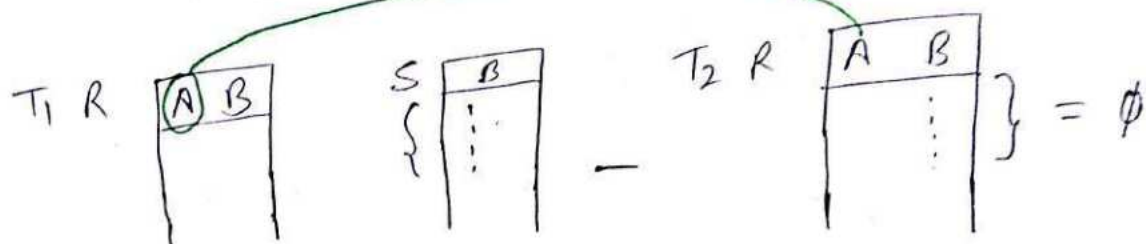
Select Cid
FROM course
where Inst = Korth
EXCEPT

Bottom ②

Select Cid
FROM Enroll T_2
where $T_2.sid = T_1.sid$

);

$$\Rightarrow \pi_{AB}(R) / \pi_B(S)$$



Query ②:-

Select Distinct A

FROM R T_1

Where NOT EXISTS (

Select B
FROM S
EXCEPT
Select B
FROM R T_2
where $T_2.A = T_1.A$

);

⇒ Retrieve sid of enroll such that no cid of Korth which is not enrolled by sid.

Query (3)

* Select distinct T₁.sid

① From Enroll T₁

⑤ where NOT EXISTS (Select *

② From Course C

where C.Inst = Korth

and NOT EXISTS (

④
Loops until all records are compared

Select *

③ From Enroll T₂

where T₂.cid = C.cid
and T₂.sid = T₁.sid

));

Query (4) Non-correlated Query :-

⇒ $\pi_{sid, cid}(E) / \pi_{cid}(\sigma_{Inst=Korth}(C))$

$$\equiv \pi_{sid}(E) - \pi_{sid} \left(\underbrace{\pi_{sid}(E) \times \pi_{cid}(\sigma_{Inst=Korth}(C))}_{Temp} - \pi_{sid, cid}(E) \right)$$

* Select sid

FROM Enroll

EXCEPT

Select sid

FROM (

Select sid, course.cid

FROM Enroll, course

where Inst = Korth

EXCEPT

Select sid, cid

FROM Enroll) temp;

* Tuple Relational Calculus [TRC] :-

- ⇒ TRC non-procedural Query Language. [Informal lang.]
- ⇒ TRC uses predicate calculus & First order Logic formulas.

* Basic Formulas:-

Simple stmts which are True or False

P, Q are 'Predicate' statements.

x is tuple variable

- $P \wedge Q$
- $P \vee Q$
- $\neg P$
(not)
- $P \rightarrow Q$
- $x \in \text{Relation}$
- Quantifiers ($\exists, \forall$)

→ $\exists x \in \text{Rel}(P(x))$: TRUE if some tuple (x) belongs to rel such that $P(x)$ is true.

→ $\forall x \in \text{Rel}(P(x))$: TRUE if every tuple (x) belongs to rel such that $P(x)$ is true.

* Relational Algebra $\left\{ \pi, \sigma, \times, \rho, \cup, -, \bowtie, \cap, / \right\}$

* Relational Calculus $\left\{ \vee, \wedge, \neg, \rightarrow, \in, \exists, \forall \right\}$

* Format of TRC Query :-

 $\{ T / P(T) \}$

T : Tuple Variable

$P(T)$: Formula over tuple variable

Retrieves set of tuples (T)
those are satisfied $P(T)$

Eg:-

 $\{ T / T \in \text{Stud} \wedge T.\text{age} > 20 \}$ $= \sigma_{\text{age} > 20}(\text{Stud})$

Retrieves students whose age more than 20.

* FREE tuple Variable :-

$\Rightarrow$ Not bounded by quantifier

$T \in \text{Stud} \begin{cases} \rightarrow \text{True} \\ \rightarrow \text{False} \end{cases}$



T : Universe of Tuples

* Bounded Tuple Variable :-

$\Rightarrow$ Variable bounded by Quantifier ($\exists, \forall$)

 $\exists T_1 \in \text{Stud} (P(T_1))$ $\forall T_2 \in \text{Stud} (P(T_2))$

Ego:-

$$\exists_x \underbrace{(x > 10)}_{x=12} \wedge \exists_x \underbrace{(x : \text{Prime})}_{x=7}$$

$$\exists_x \underbrace{(x > 10 \wedge x = y)}_{\substack{\downarrow \\ x=11}} \wedge \exists_x \underbrace{(x : \text{Prime} \wedge x = y)}_{\substack{\downarrow \\ x=11}}$$

*) Supplier (Sid Sname Rating City)

S1
S2
S3
S4
S5

Parts (Pid Pname Color)

P1	-	Red
P2	-	Red
P3	-	Green
P4	-	Green
P5	-	Blue

Catalog (Sid Pid Cost)

S1	P1	60
S1	P2	80
S1	P3	50
S2	P2	70
S2	P4	60
S3	P2	60
S4	P4	50

1) Retrieve Sid's supplied some red part.

Soln

$$\pi_{\text{sid}} \left(\text{Catalog} \bowtie \sigma_{\text{parts}} \left(\text{col} = \text{Red} \right) \right)$$

$$\Rightarrow \left\{ T / \exists T_1 \in \text{Catalog} \exists T_2 \in \text{Parts} \right. \\ \left. \left(T_1.\text{Pid} = T_2.\text{Pid} \wedge T_2.\text{Col} = \text{Red} \wedge T = T_1.\text{sid} \right) \right\}$$

↑
Sid's

(OR)

$$\Rightarrow \left\{ T / T \in \text{Catalog} \wedge \exists T_2 \in \text{Parts} (T_2.\text{Col} = \text{Red} \wedge T_2.\text{Pid} = T.\text{Pid}) \right\}$$

* Set Operations :-

- $R \cup S = \{ x / x \in R \overset{\text{OR}}{\vee} x \in S \}$
- $R - S = \{ x / x \in R \underset{\text{But NOT}}{\wedge} \neg(x \in S) \}$
- $R \cap S = \{ x / x \in R \underset{\text{AND}}{\wedge} x \in S \}$

11) Retrieve Sid's supplied some red part or green part.

Soln

$$\pi_{\text{sid}} \left(\text{Catalog} \bowtie \sigma_{\text{parts}} \left(\text{Col} = \text{Red} \vee \text{Col} = \text{Green} \right) \right)$$

$$\Rightarrow \left\{ T / \exists T_1 \in \text{Catalog} \exists T_2 \in \text{Parts} \right. \\ \left. \left(T_1.\text{Pid} = T_2.\text{Pid} \wedge (T_2.\text{Col} = \text{Red} \vee T_2.\text{Col} = \text{Green}) \right. \right. \\ \left. \left. \wedge (T = T_1.\text{sid}) \right) \right\}$$

(OR)

$$\Rightarrow \left\{ T / \exists T_1 \in \text{Catalog} \exists T_2 \in \text{Parts} \right. \\ \left. (T_1.\text{pid} = T_2.\text{pid} \wedge T_2.\text{col} = \text{Red} \wedge T = T_1.\text{sid}) \right\}$$

sid : some red part

V

$$\exists T_1 \in \text{Catalog} \exists T_2 \in \text{Parts} \\ (T_1.\text{pid} = T_2.\text{pid} \wedge T_2.\text{col} = \text{Green} \wedge T = T_1.\text{sid}) \left. \vphantom{\exists T_1 \in \text{Catalog} \exists T_2 \in \text{Parts}} \right\}$$

sid : some green part

iii) Retrieve sid's of supplier whose rating maximum.

Some

$$(\geq \text{every}) \equiv \left\{ \text{sid's of suppliers} \right\} - \left\{ \text{sid's whose rating} < \text{some rating of supplies} \right\}$$

$$\pi_{\text{sid}}(\text{supplier}) - \pi_{\text{sid}} \left(\text{supplier} \bowtie_{\text{rating} < R} \rho(\text{supplier}) \right)$$

$\mathbb{Z}, \mathbb{N}, \mathbb{R}$

$$\Rightarrow \left\{ T / \exists T_1 \in \text{supplier} (T = T_1.\text{sid}) \wedge \right.$$

$$\neg \exists T_1 \in \text{supplier} \exists T_2 \in \text{supplier}$$

$$\left. (T_1.\text{rating} < T_2.\text{rating} \wedge T = T_1.\text{sid}) \right\}$$

(OR)

$$\Rightarrow [\geq \text{every}]_{(\text{max})} \quad \exists x \forall y (x \geq y)$$

$$\left\{ T / \exists T_1 \in \text{supplier} \forall T_2 \in \text{supplier} \right.$$

$$\left. (T_1.\text{rating} \geq T_2.\text{rating} \wedge T = T_1.\text{sid}) \right\}$$

sid's of suppliers

- Select Sid
FROM Supplier
where rating $\geq$ ALL (Select rating
FROM Supplier);

iv) Retrieve supplier^{Sid's} who supplied only one part.

Solⁿ
Sid's supplied only one part $\equiv$ Sid's supplied some part $-$ Sid's supplied at least two parts

$$\pi_{\text{sid}}(\text{Catalog}) - \pi_{\text{sid}} \left(\text{Catalog} \bowtie_{\substack{\text{sid} = S \\ \wedge \text{pid} \neq P}} \rho(\text{Catalog})_{S, P, C} \right)$$

$$\Rightarrow \left\{ T / \exists T_1 \in \text{Catalog} (T = T_1.\text{sid}) \wedge \right. \\ \left. \neg \exists T_1 \in \text{Catalog} \exists T_2 \in \text{Catalog} \right. \\ \left. (T_1.\text{sid} = T_2.\text{sid} \wedge T_1.\text{pid} \neq T_2.\text{pid} \wedge T = T_1.\text{sid}) \right\}$$

Sid's supplied some part

Sid's supplied at least two parts

* Unsafe TRC Query:-

$\Rightarrow$ TRC Query whose result infinite records

Eg^o 1) $\left\{ T / \neg (T \in \text{Suppliers}) \right\}$: Unsafe TRC Query

Set of tuples those are not belong to Supplier



T: Universe of Tuples

[Infinite set of tuples]

$$2) \quad T / \neg (\exists T_1 \in \text{Supplier} (T_1.\text{rating} > 10 \wedge T = T_1.\text{sid}))$$

Unsafe TRC Query.

* Expressive Power Comparison :-

$$\left\{ \begin{array}{l} \text{Basic RA} \\ \text{Queries} \\ \text{expressive power} \end{array} \right\} = \left\{ \begin{array}{l} \text{Safe TRC} \\ \text{Queries} \\ \text{expressive power} \end{array} \right\} = \left\{ \begin{array}{l} \text{Safe DRC} \\ \text{Queries} \\ \text{expressive power} \end{array} \right\}$$

DRC = Domain Relational Calculus

* Basic RA Queries :-

⇒ RA Queries which can formulate (express) using

$$\left\{ \begin{array}{l} \pi, \sigma, \rho, \times, \cup, -, \cap, \\ \bowtie, \bowtie_c, \bowtie_{\neq}, \bowtie_{\leq}, \bowtie_{\geq}, \div \end{array} \right\}$$

* Queries failed to express by using ^{Safe TRC} basic RA :-

⇒ Count of Attribute values/records

[Query required length [Count] which is not given in Question]

⇒ Sum, Avg of Attribute values

⇒ Ordering of records in Asc/Desc based on attribute values.

⇒ To test duplicate records in relation/result.

⇒ Insertion/Deletion/Update of records.

Eg:- family (parent, child, childDOB)

1) Retrieve all descendants of "Amy".

⇒

Parent	Child
Amy	A
Amy	B
A	C
A	D
E	F
D	G
G	H
H	Bob

Not fixed
length RA
Query

↑↑
Not possible to
express using
Basic RA

$$\pi_{\text{child}} \left(\sigma_{\text{family}} \left(\begin{array}{l} \text{parent} = \text{Amy} \end{array} \right) \right) \cup$$

$$\pi_{T_2.\text{child}} \left(\sigma_{\left(\rho(T_1.\text{family}) \times \rho(T_2.\text{family}) \right)} \left(T_1.\text{parent} = \text{Amy} \wedge T_1.\text{child} = T_2.\text{parent} \right) \right) \cup$$

$$\pi_{T_3.\text{child}} \left(\sigma_{\left(T_1 \times T_2 \times T_3 \right)} \left(T_1.P = \text{Amy} \wedge T_1.C = T_2.P \wedge T_2.C = T_3.P \right) \right) \cup$$

⋮

$$\pi_{T_n.\text{child}} \left(\sigma_{\left(T_1 \times T_2 \times \dots \times T_n \right)} \left(T_1.P = \text{Amy} \wedge T_1.C = T_2.P \wedge \dots \wedge T_{n-1}.C = T_n.P \right) \right) \cup$$

11) Retrieve all great grand parents of Bob.

Ans Not possible to express using basic RA ⇒ Not fixed
length RA Query.